Preprints.org

Concept Paper

Not peer-reviewed version

# The Architectural Framework of Agentic AI 2.0

Quy Minh Le , Thai Hoa Tran , Duc-Hai Nguyen , Duy Khuong Nguyen , Phong X. Nguyen , Duc Trong Le , Barry O'Sullivan , Hoang D. Nguyen *

Posted Date: 10 October 2026

Keywords: Agentic AI; foundation models; multi-agent systems; reference architecture; architectural analysis; computational continuity; AI governance

Preprints.org is a free multidisciplinary platform providing preprint service that is dedicated to making early versions of research outputs permanently available and citable. Preprints posted at Preprints.org appear in Web of Science, Crossref, Google Scholar, Scilit, Europe PMC, OpenAlex.

Copyright: This open access article is published under a Creative Commons CC BY 4.0 license, which permit the free download, distribution, and reuse, provided that the author and preprint are cited in any reuse.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1

Disclaimer/Publisher's Note: The statements, opinions, and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions, or products referred to in the content.

*Concept Paper*

# The Architectural Framework of Agentic AI 2.0

**Quy Minh Le [1,†], Thai Hoa Tran [1,†], Duc-Hai Nguyen [2,†], Duy Khuong Nguyen [3], Phong X. Nguyen [3], Duc Trong Le [4], Barry O'Sullivan [2] and Hoang D. Nguyen [2,*]**

[1] FPT Software, FPT Corporation, Vietnam
[2] School of Computer Science & IT, University College Cork, Cork, Ireland
[3] Quantum AI & Cybersecurity Institute, FPT Corporation, Vietnam
[4] Vietnam National University, Vietnam
* Correspondence: hn@cs.ucc.ie
† Quy Minh Le, Thai Hoa Tran, and Duc-Hai Nguyen contributed equally to this work.

**Abstract**

Maintaining coherent agentic work becomes an architectural challenge as information, participants, commitments, execution components, and operating conditions change. This paper proposes *Agentic AI 2.0*, a system-level architectural framework that integrates established concepts to organize ongoing goal-directed work in foundation-model-based systems. Six constitutive properties characterize this organization: Maintained Process State, Collective Coordination, Shared Operational Knowledge, Governed and Grounded Action, Computational Continuity, and Feedback-Driven Adaptation. The framework connects these properties to role-bearing agents, shared operational objects, interface contracts, and interacting execution, coordination, and adaptation loops. A composition guard specifies how local acceptance decisions connect across operational boundaries, while a criterion-level codebook supports configuration-specific assessment from explicit evidence. We examine the framework through a constructed production-order trace, disruption and handoff branches, counterexamples, and static source profiles of three application configurations and one infrastructure boundary case. The analyses distinguish local feasibility from joint commitments, effect reconciliation from execution-right control, and retained skill versions from governed activation and reuse. They also separate scoped negative evidence from unresolved evidence when diagnosing architectural relationships. Candidate evaluation and later reuse remain conditional in the worked example. Prospective profiles for enterprise productivity, adaptive manufacturing, and distributed mobility identify domain-specific evidence needs and evaluation priorities. The contribution is a traceable basis for architectural analysis, comparison, and design. Independent assessor studies, comparative diagnosis, and controlled and longitudinal implementation studies are needed to establish its practical value and the relationship between architectural choices, operational outcomes, and cost.

**Keywords:** Agentic AI; foundation models; multi-agent systems; reference architecture; architectural analysis; computational continuity; AI governance

## 1. Introduction

Foundation models support agents that interpret goals, reason over intermediate steps, use tools, and interact with external environments. ReAct connects reasoning and action through environmental observations [1], while Cognitive Architectures for Language Agents (CoALA) organizes memory, actions, and decision processes [2]. Computer-use benchmarks extend this setting to desktop applications and tasks combining graphical interaction with command-line and code execution [3,4]. Maintaining work across these interactions introduces architectural questions about state, responsibility, and the evidence on which subsequent decisions depend.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



### 1.1. From Task Execution to Ongoing Work

Consider a production order involving planning, procurement, machine operation, and quality control. A machine failure may invalidate an accepted allocation. An executor may become unavailable after issuing an action whose external effect remains uncertain. Another executor must then determine which obligations remain and whether dependent work can resume. Experience from the disruption may inform a later order, subject to the conditions under which that experience applies.

We use *ongoing autonomous work* for goal-directed work that can span pauses, authorized human decisions, changes in responsible actors, and recovery from disruption. Its organization must preserve the information and responsibilities needed for subsequent decisions. Role-based workflows, retained memory, and reusable skills provide relevant mechanisms [2,5,6]. Their contribution depends on how they connect to the same undertaking across interactions and execution episodes.

### 1.2. Architectural Concern and Contributions

The architectural concern is the connection between capability outputs and their operational consequences. Observations require acceptance before they change maintained state; proposed plans require authority and compatible commitments before they allocate work. Execution requests require environmental evidence to establish their outcomes. Component changes require reconciliation and authority to continue, while reusable changes require evaluation and context-specific adoption.

Collaborative planning and normative multi-agent systems provide foundations for commitments, roles, permissions, and obligations [7–9]. Distributed recovery addresses the relationship between restored computation and prior external effects [10]. We bring these concerns together around the records and acceptance responsibilities of ongoing foundation-model-based work.

This paper proposes *Agentic AI 2.0*, a system-level architectural framework connecting accepted state, commitments, authority, outcome evidence, continuation, and versioned change. The designation identifies an architectural orientation rather than a chronological generation or maturity level. Its unit of analysis is a declared configuration and operational scope, including the actors, execution components, interfaces, authority arrangements, and changes relevant to the assessed work.

The paper makes three contributions:

1. **Architectural characterization and profiling.** Six constitutive properties define the proposed family. A configuration-specific codebook distinguishes established relationships, scoped negative witnesses, and unresolved evidence, with explicit rules for deriving a property-level profile.
2. **Reference architecture.** Role-bearing agents, shared operational objects, interface contracts, and execution, coordination, and adaptation loops specify how the relevant responsibilities connect. Acceptance conditions identify when information, proposals, actions, and changes acquire an operational status.
3. **Architectural analysis.** Comparison with close predecessors, a constructed production-order trace, witness requirements, changed-condition checks, counterexamples, and implementation-source profiles make selected relationships inspectable. Diagnostic findings identify affected decisions, supporting evidence, and follow-up checks; application profiles examine their implications in three domains.

The analysis supports examination of the proposed distinctions within its declared cases and evidence. Independent profiling reliability, comparative diagnostic value, and effects on operational performance form the evaluation agenda in Section 7.

Sections 2 and 3 establish the positioning, definitions, and profiling rules. Sections 4–5 develop the mechanisms and architecture. Sections 6–8 present the analysis, application profiles, discussion, and conclusions.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



## 2. Architectural Foundations and Positioning

The framework draws on research in agent organization, maintained work state, conditional progression, recovery, and adaptation. This targeted comparison identifies the abstractions inherited by the synthesis and the relationships examined through its operational perspective.

### *2.1. Architectural Foundations for Ongoing Work*

Cognitive architectures organize memory, reasoning, and learning within an agent. Soar, for example, combines working memory and problem-space reasoning with chunking [11]. Distributed cognition extends the analytical boundary to information use across people, artifacts, and external representations [12]. These perspectives inform the distinction between an individual agent's retained information and knowledge used across operational responsibilities.

Organizational and collaborative approaches provide direct precedents for responsibility allocation and commitments. Gaia models roles, permissions, interactions, and organizational rules [13]; SharedPlans formalizes partial plans, allocated actions, and commitments to joint activity [7]. STEAM connects joint intentions to communication, responsibility monitoring, and team reorganization [8]. Electronic institutions and normative multi-agent systems address protocols, obligations, permissions, and enforcement [9,14].

Work-centered models connect maintained information to progression conditions. Guard-Stage-Milestone (GSM) couples business-entity information with guards, stages, and milestones, while adaptive case management examines changes to artifact-centric behavior and preservation conditions [15,16]. Autonomic computing contributes component interfaces and feedback-based management: White et al. [17] describe interacting autonomic components and prototype studies, and IBM's blueprint organizes monitor, analyze, plan, and execute functions around shared knowledge [18].

Recovery adds requirements concerning external effects. Rollback-recovery protocols account for communication dependencies and outputs beyond the recovering computation [10]. Chubby's recipient-checked sequencers illustrate control of stale execution rights at the protected interface [19]. Workflow patterns, process mining, and continual learning contribute control-flow structures, analysis of execution records, and retention and transfer under change, respectively [20–22].

Table 1 identifies the closest architectural precedents and the connections developed in this paper.

**Table 1.** Close architectural predecessors and their relation to the proposed reference organization.

| Predecessor and unit | Inherited abstraction | Connection examined here |
|---|---|---|
| Gaia; electronic institutions; normative systems [9,13,14] | Roles, responsibilities, permissions, protocols, and organizational rules. | Actor assignments and authority linked to the state, decisions, and evidence of identified work. |
| SharedPlans and STEAM [7,8] | Collaborative plans, commitments, monitoring, and team reorganization. | Commitment revision connected to affected obligations, environmental outcomes, and continuation. |
| GSM and adaptive case management [15,16] | Maintained business entities, conditional progression, and controlled lifecycle change. | Work identity and accepted terms connected to actor authority, uncertain effects, and reusable changes. |
| Autonomic computing [17,18] | Component interfaces, shared knowledge, and feedback-based management. | Acceptance responsibilities across execution, coordination, and adaptation. |
| Rollback recovery and Chubby [10,19] | Recovery consistent with external effects; recipient-checked control of stale execution rights. | Effect reconciliation and execution-right control connected to responsibility and authority to continue. |

The comparison identifies conceptual antecedents. Property-level support requires assessment of a selected configuration under Section 3.3. Section 2.4 considers a composition of these predecessors as an analytical comparator.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



### 2.2. Contemporary Agentic Mechanisms and Evidence

Contemporary agents connect perception, reasoning, action, and learning through several mechanisms. ReAct interleaves reasoning with actions and observations, while Language Agent Tree Search combines search, value estimates, reflection, and external feedback [1,23]. CoALA provides a memory-and-action account of the individual language agent [2]. AutoGen supports programmable conversations among models, tools, and humans [24]. CAMEL studies role-playing cooperation [25], while MetaGPT organizes software development through role-specific procedures and intermediate artifacts [5].

Computer-use benchmarks connect agent behavior to environmental outcomes. OSWorld includes desktop and web tasks with execution-based evaluation [3]. WeaveBench examines hybrid graphical user interface (GUI)–command-line interface (CLI)/code workflows and trajectory-aware evaluation [4]. Other benchmarks address policy-following reliability and prompt-injection robustness [26,27]; evaluation methodology also considers repeated-trial reliability and the cost of obtaining a result [28].

Reflexion retains verbal reflections [29], while Voyager accumulates executable skills from environmental feedback [6]. Agent Workflow Memory derives reusable workflows [30], and A-Mem revises a linked memory network [31]. Agentic reinforcement learning (RL) uses multi-step interaction data for policy updates [32,33]. These mechanisms provide different objects of adaptation, with distinct conditions for their subsequent operational use.

Recent continuity preprints examine checkpoint/resume contracts [34], security-relevant rollback dependencies [35], transactional tool progress [36], durable workflow recovery [37], and event-sourced handoff across coding agents [38]. Related work addresses persistent memory, governance, and longer-horizon reliability [39,40]. Together, these studies motivate assessment of how restored state, external effects, outstanding commitments, and authority connect within a supported continuation path.

Empirical studies also motivate attention to process behavior. Cemri et al. [41] derive MAST from 150 traces and release over 1,600 annotated traces across seven frameworks, identifying 14 failure modes in system design, inter-agent misalignment, and task verification. Their reported failure rates apply to the evaluated framework–benchmark configurations. Rabanser et al. [42] examine 15 models on GAIA and $\tau$-bench and report limited reliability gains relative to task-performance improvements. These findings motivate architectural questions about coordination and verification; the validity of the proposed profiling procedure remains a separate evaluation question.

### 2.3. Agentic Operating Systems and Shared Infrastructure

We use *Agentic Operating System* (Agentic OS) for infrastructure supplying shared services to agents. AIOS provides agent-oriented kernel services, while Borg supplies a precedent for cluster resource management [43,44]. The Model Context Protocol (MCP) specifies tool discovery and invocation, and Agent2Agent (A2A) specifies agent discovery and task-oriented interaction [45,46]. Other proposals address identifiers, certification, rollback, and broader operating infrastructure [47,48].

These services can support the reference architecture through scheduling, storage, access control, and execution management. Their operational significance depends on the application responsibilities and acceptance conditions they implement. Resource capacity and technical permissions must therefore be related to the commitments and authority of the assessed work.

### 2.4. Positioning and Scope of the Architectural Synthesis

The synthesis organizes inherited abstractions around the same ongoing foundation-model-based work. Shared operational objects connect its state, commitments, authority, actions, and changes; interface contracts identify the conditions for changing their operational status. Tables 3 and 4 develop this reference organization.

A composition of artifact lifecycles, organizational agent models, recovery and execution-right controls, and policy-governed adaptation provides a substantive analytical comparator. Such a

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



composition could realize the proposed responsibilities. The question examined here is how these responsibilities connect: an Action Record refers to the work, plan version, executor, authority, and outcome evidence; handoff accounts for that action and the remaining commitments; and a reusable change identifies its accepted version and applicable work. Execution-right control, responsibility transfer, and change applicability each require evidence within this connected account.

The comparison follows each source's unit of analysis and evidence scope. Publication status is recorded separately from evidence type: formal results concern their models and assumptions, implementations their identified configurations, evaluations their tested conditions, and technical specifications their interface scope. Author-reported findings and independent validation remain distinguishable. Unreported relationships are evidence gaps.

Section 3.2 explains the retrospective construction of the six properties from these antecedents. The worked example and configuration profiles examine selected consequences of the proposed organization. Relative expressive power and comparative diagnostic accuracy require the independent comparisons outlined in Section 7.

## 3. Framework and Architectural Profiling

The framework characterizes how a bounded configuration organizes ongoing work. Its application requires explicit definitions of the undertaking, participating actors, supported changes, and evidence used for assessment.

### *3.1. Definition and Unit of Analysis*

**Definition 1** (Agentic AI 2.0)**.** *Agentic AI 2.0 denotes a system-level architectural organization in which ongoing goal-directed work is maintained through explicit process state, coordinated across participating agent roles, supported by shared operational knowledge, and advanced through governed and grounded action. The organization supports continuation across specified changes in execution components and uses feedback to guide controlled adaptation and the accumulation of reusable experience.*

A *model* supplies cognitive capabilities for interpretation, reasoning, planning, or action selection. A *harness* connects these capabilities to context, memory, tools, execution control, validation, guardrails, and tracing as required by the operational scope. A *runtime* provides their software execution environment. These functions may share a package or be distributed across services.

A *role* specifies an expected contribution, responsibilities, and a decision or action scope. An *operational actor* is an identifiable bearer of those responsibilities. An *agent* is a software operational actor with an assigned role; humans may also hold operational responsibilities and authority. An actor boundary requires a separately attributable assignment, a defined decision or action scope, and traceable consequences of its contributions, such as changes to accepted state, commitments, actions, or another actor's evidence. A planner, critic, dispatcher, or store operating within another actor's assignment remains an internal or supporting component.

These mappings are many-to-many: an actor may perform several roles and use multiple models or services, while several actors may share a model and runtime. Actors may contribute sequentially when their separate assignments and interdependencies remain identifiable. Collective Coordination requires distinct *agent actors*; human participation is assessed through the information, authority, or decisions it contributes. The configuration may operate within an organization or another environment.

A *process* denotes organized goal-directed work, including its state, dependencies, commitments, and completion conditions, rather than an operating-system process. A *work episode* is a bounded undertaking that may span several runtime execution episodes. *Process identity* associates those episodes with a stable reference, or an explicit mapping between references, accepted terms, and authorized revision history. An amendment remains within the same undertaking when the declared identity rule recognizes it and records its consequences for existing obligations. A different undertaking is recorded as a successor, with its predecessor relationship and the disposition of inherited or outstanding obligations.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



*Execution lineage* records derivation among runs, checkpoints, branches, or configurations. *Valid continuation* establishes whether the identified process may proceed under its state, commitments, evidence, compatibility conditions, responsibility, and authority. Identity and lineage support traceability; admissibility of the next transition requires these continuation conditions.

An *agent lifetime* covers the actor's existence and operational history under the applicable identity and versioning rules. A *process lifetime* runs from establishment to completion or authorized termination, including pauses and changes in participation. One process may span several agent lifetimes, and an agent may contribute to several work episodes.

The unit of analysis is a *specific system configuration within a declared operational scope, with identified versions and specified changes under consideration*. The scope covers the workload, actors and roles, models, harnesses, runtimes, shared services, environment interfaces, authority arrangements, and environmental assumptions relevant to the work. Here, ongoing work includes suspension, handoff, and authorized termination. Section 3.3 specifies the evidence needed to characterize this unit.

### *3.2. Constitutive Architectural Properties*

The six properties specify the requirements adopted for this architectural family. Their construction is a *retrospective analytical reconstruction* of the targeted comparison in Section 2.4. The source abstractions are interpreted as relations among work, information, responsible actors, and the conditions governing operational use or change. A concept is retained at the property level when it identifies a work-level responsibility with a distinguishable assessment question; retrieval, tool invocation, checkpointing, and model updating are mechanisms that can support those responsibilities.

The properties are interdependent, and their minimality and exhaustiveness remain open to evaluation. Table 2 summarizes their assessment questions. Appendix A gives the constituent criteria, grouping alternatives, and boundary cases.

#### 3.2.1. Maintained Process State

The system maintains an identifiable account of accepted goals and completion conditions, progress, dependencies, unfinished obligations, and relevant environmental conditions across execution episodes. State-acceptance and version rules distinguish received information from the current operational account. Uncertainty and unresolved conditions remain visible where they affect subsequent decisions. This property concerns the accepted condition of the undertaking and its use in determining what remains to be done.

#### 3.2.2. Collective Coordination

Distinct agent actors in interdependent roles make attributable contributions that affect planning, allocation, or revision of the same work. Coordination allocates responsibilities, manages dependencies and resource conflicts, and establishes or revises commitments. A *commitment* records an accepted undertaking attributed to a responsible actor through its current role assignment, with conditions governing its application or revision. Assessment examines how conflicting proposals and changed conditions lead to authorized dispositions of affected plans and commitments.

#### 3.2.3. Shared Operational Knowledge

Information is accumulated, retrieved, and made usable across distinct operational actors under contextual, access, and validity conditions. Consumers must be able to distinguish observations, assumptions, knowledge claims, decisions, and outcomes, together with relevant provenance, temporal context, and versions. Conflict and supersession rules preserve these distinctions during use. Linked stores and role-specific views may provide this support. Maintained Process State identifies the accepted situation of an undertaking; Shared Operational Knowledge concerns information used across responsibilities to interpret and act within it.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



#### 3.2.4. Governed and Grounded Action

Actions proceed under applicable authority, preconditions, and constraints, through interfaces connected to the declared environment. Outcome acceptance uses evidence attributable to the requested operation. Authority to act, dispatch admission, and confirmation of an effect are separate judgments attached to the same action. Failure and uncertainty remain explicit when subsequent decisions concern waiting, investigation, retry, escalation, or dependent execution.

#### 3.2.5. Computational Continuity

The configuration supports valid continuation of identified work across specified interruptions or changes to agents, models, harnesses, runtimes, or shared infrastructure. Continuation accounts for accepted terms, unfinished commitments, prior effects, and the receiving configuration's compatibility. Responsibility and authority are retained, transferred, or re-established, with overlapping execution rights resolved. The claim is bounded by the named changes and their reconciliation, compatibility, and authority conditions.

#### 3.2.6. Feedback-Driven Adaptation

Feedback informs candidate changes or retained experience intended for later work. Candidates undergo evaluation against declared acceptance conditions; activation identifies the accepted version, applicability, and affected work and dependencies. Reuse links the retained version and application context to outcome feedback. Changes may concern knowledge, skills, workflows, planning policies, or model parameters. Evaluation may support adoption, revision, rejection, or withdrawal. Current-work plan repair and cross-work reuse remain distinct, and performance benefits require evidence beyond the existence of this pathway.

**Table 2.** Architectural properties and assessment questions.

| Property | Assessment question | Evidence to examine |
|---|---|---|
| **P1** Maintained Process State | What is the accepted condition of the undertaking across execution episodes? | Work references, accepted terms, state use, uncertainty, update decisions, and versions. |
| **P2** Collective Coordination | How do interdependent agent contributions establish and revise shared commitments? | Actor assignments, allocations, dependencies, conflict resolution, and commitment dispositions. |
| **P3** Shared Operational Knowledge | What information can another responsible actor use, and under which conditions? | Contributors and consumers, operational use, provenance, status, validity, and access rules. |
| **P4** Governed and Grounded Action | What authorizes the action, admits its execution, and establishes its outcome? | Authority, preconditions, interfaces, request-linked evidence, and outcome dispositions. |
| **P5** Computational Continuity | Under what conditions may identified work continue across a specified change? | Reconciliation, compatibility, outstanding obligations, responsibility, authority, and execution-right control. |
| **P6** Feedback-Driven Adaptation | How does feedback become evaluated change and governed reuse in later work? | Candidate versions, evaluation, acceptance, activation, dependencies, reuse context, and outcome feedback. |

The questions summarize the proposed architectural requirements. Coding follows the complete criteria in Tables A1 and A2; examples admit equivalent mechanisms. Support is scoped to the claim basis in Section 3.3.

Figure 1 presents structural and temporal views of these relationships. The panels describe complementary aspects of the work; their conceptual groupings permit different actor counts, topologies, and deployment arrangements.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



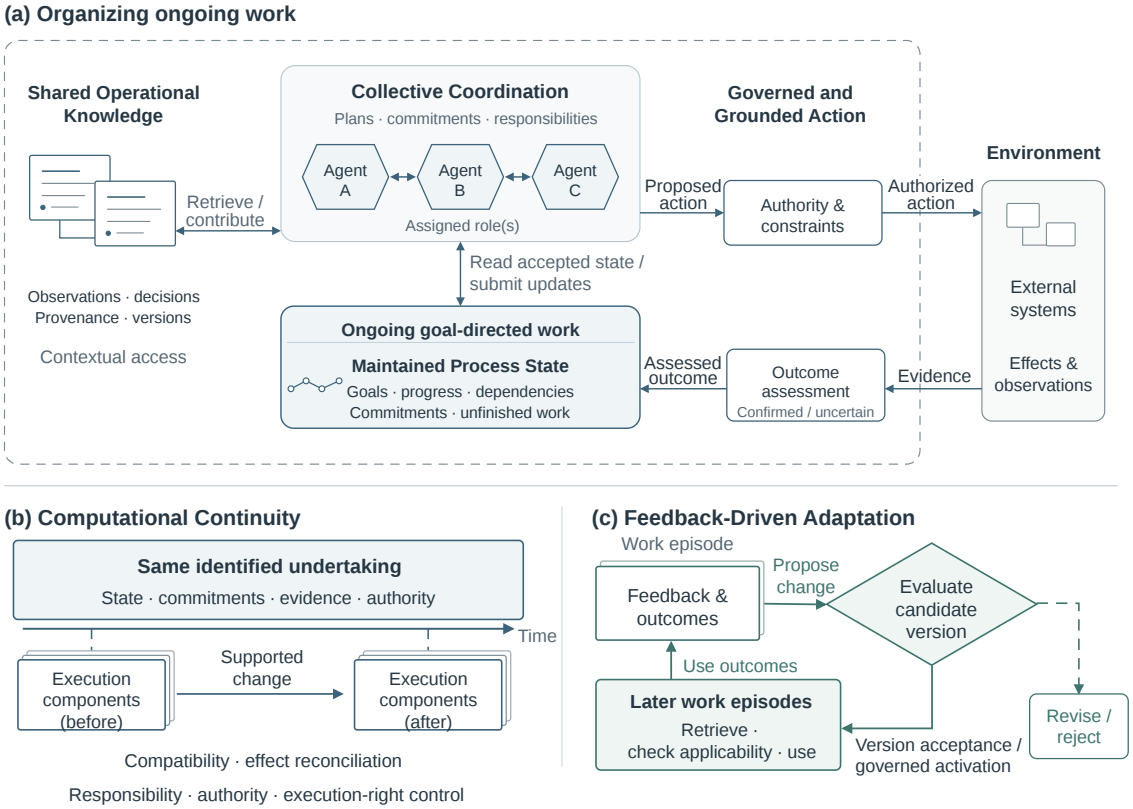



**Figure 1.** Constitutive properties of Agentic AI 2.0. (a) Distinct agent actors coordinate ongoing work through maintained state, commitments, and shared knowledge; action admission and outcome assessment connect this organization to the environment. (b) Continuation of identified work across supported component changes requires compatibility, effect reconciliation, responsibility, authority, and control of overlapping execution. (c) Feedback informs versioned candidates, evaluation, acceptance, and governed activation for applicable work; reuse outcomes inform subsequent adaptation. Arrows show selected relationships and enclosures denote conceptual groupings.

### *3.3. Architectural Profiles and Transitional Systems*

Assessment first declares the workload and work-episode boundaries, configuration versions and enabled components, actor assignments, interfaces and external services, authority and trust assumptions, supported changes, excluded conditions, and available evidence. External services contribute when their applicable contracts and evidence are identified; dependencies remain part of the assessment even when supplied outside the deployment boundary. Computational Continuity requires at least one named interruption or change and its continuation conditions.

Judgments follow operational relationships. Repackaging equivalent responsibilities, decision scopes, and dependencies leaves a profile unchanged when its evidence is equivalent. Changes in the configuration, assessment boundary, available evidence, or codebook are recorded separately so that reassessment can identify what changed.

Codebook v1.0 comprises 21 criteria in Appendix A: three each for P1–P3 and four each for P4–P6. Each record retains the criterion identifier, scope, applicable conditions, source and version, evidence, counterevidence, unresolved questions, code, and rationale. The *claim basis* is recorded independently of the code:

- **D: specification**—an explicitly described responsibility or procedure;
- **I: implementation**—its realization in an identified configuration;
- **O: observed behavior**—particular executions under stated conditions;
- **V: evaluated behavior**—behavior assessed under an identified protocol, cases, and reported outcomes.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



Each basis bounds the conclusion it supports. Direct source inspection, author-reported implementation, and execution by the assessor retain their respective provenance. Separate bases may yield separate profiles for the same configuration.

Each criterion receives one code:

(i) **Established (E):** admissible evidence establishes the complete criterion at the declared basis and scope. Evidence for only some constituent obligations is retained as a narrower finding.

(ii) **Not satisfied (N):** admissible evidence establishes an excluded, omitted, or violated required relationship within scope. The record identifies the failed obligation and a configuration-matched negative witness; equivalent mechanisms are assessed by the relationship they realize.

(iii) **Unknown (U):** evidence establishes neither E nor N. This includes missing information, unexamined conditions, uncertain source applicability, or unresolved material source conflicts.

Admissible evidence addresses the same criterion, configuration, version, conditions, and claim basis. A confirmed in-scope counterexample yields N even when other included cases have positive evidence. Unresolved disagreement about source applicability or the validity of a counterexample yields U. A rejection or suspended handoff can establish support when that disposition is required by the criterion.

All criteria are required for complete characterization. Applicability qualifies condition-dependent obligations within a criterion: a conflict-handling branch may be unnecessary when its trigger is demonstrably excluded, while the remaining coordination requirements still apply. Positive relationships, including cross-actor use and continuation across a named change, require affirmative evidence; an empty declared domain cannot establish them. Unknown applicability remains an evidence gap.

For property $i$, let $R_i$ be its nonempty set of codebook criteria, and let $e_i$, $n_i$, and $u_i$ count its E, N, and U codes, respectively, with $m_i = |R_i| = e_i + n_i + u_i$. The property label is

$$L_i = \begin{cases} \text{NS}, & n_i > 0, \\ \text{S}, & e_i = m_i, \\ \text{PS}, & n_i = 0,\ e_i > 0,\ u_i > 0, \\ \text{II}, & u_i = m_i. \end{cases} \tag{1}$$

The cases are mutually exclusive and exhaustive. S denotes *Supported within scope*; NS, *Not satisfied within scope*; PS, *Partially supported by available evidence*; and II, *Insufficient information*. This is a logical aggregation of evidence, rather than a coverage percentage or maturity score.

NS records at least one unmet requirement and may coexist with established mechanisms. PS records partial knowledge with at least one E and no N. For example, a handoff that retains state but demonstrably drops an outstanding commitment can establish P5.1 and violate P5.2, yielding NS. If commitment handling is unreported, P5.2 remains U and the property is PS provided no criterion receives N. All four continuity criteria must receive E for S.

One source may support several criteria when each use has its own rationale. A supported profile must draw on compatible configurations, versions, workloads, and claim bases. For relationships between acceptance boundaries, evidence must connect the exchanged claim, dependency and version bindings, and receiving-use conditions, as specified in Section 4.4.1. A coding judgment describes that evidence; operational authorization remains a separate decision. Appendix D records the criterion vectors and source locators used in Section 6.4.

A complete Agentic AI 2.0 characterization requires S for all six properties at a compatible configuration, scope, codebook version, and claim basis. NS identifies an unmet requirement; PS and II identify incomplete support. *Transitional systems* describes configurations with established relationships alongside unresolved or unmet requirements. Where used, *Agentic AI 1.0* also denotes a stated architectural profile. These terms specify neither a development sequence nor task suitability; performance and reliability require their own evidence.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



## 4. Reference Architecture and Operational Lifecycle

The reference architecture connects the properties in Section 3 to responsibilities, operational objects, and decisions about ongoing work. Its individual, collective, and lifecycle views describe the same configuration at different levels of detail. The organization is logical: responsibilities may be combined within an implementation or distributed across agents, services, and human participants.

### *4.1. Capabilities and Functional Organization*

The seven capability families support perception, reasoning, planning, memory, predictive knowledge, action, and learning. Figure 2 connects their outputs: observations inform reasoning; plans and commitments guide action; outcome evidence informs further reasoning and experience analysis. Retained knowledge and, where used, world-model predictions support several functions.

These relations become operational through the acceptance decisions developed below. An observation carries its source and uncertainty into state assessment; a plan requires commitment acceptance; an action requires admissible dispatch; and a reusable change requires evaluation, acceptance, and context-specific activation. Capabilities can share supporting components while their outputs retain these different meanings.

Appendix B gives the supporting mechanisms, distinctions among information types, and the capability-to-property mapping. Property judgments follow Section 3.3 and Appendix A; the capability families organize supporting functions rather than additional assessment criteria.

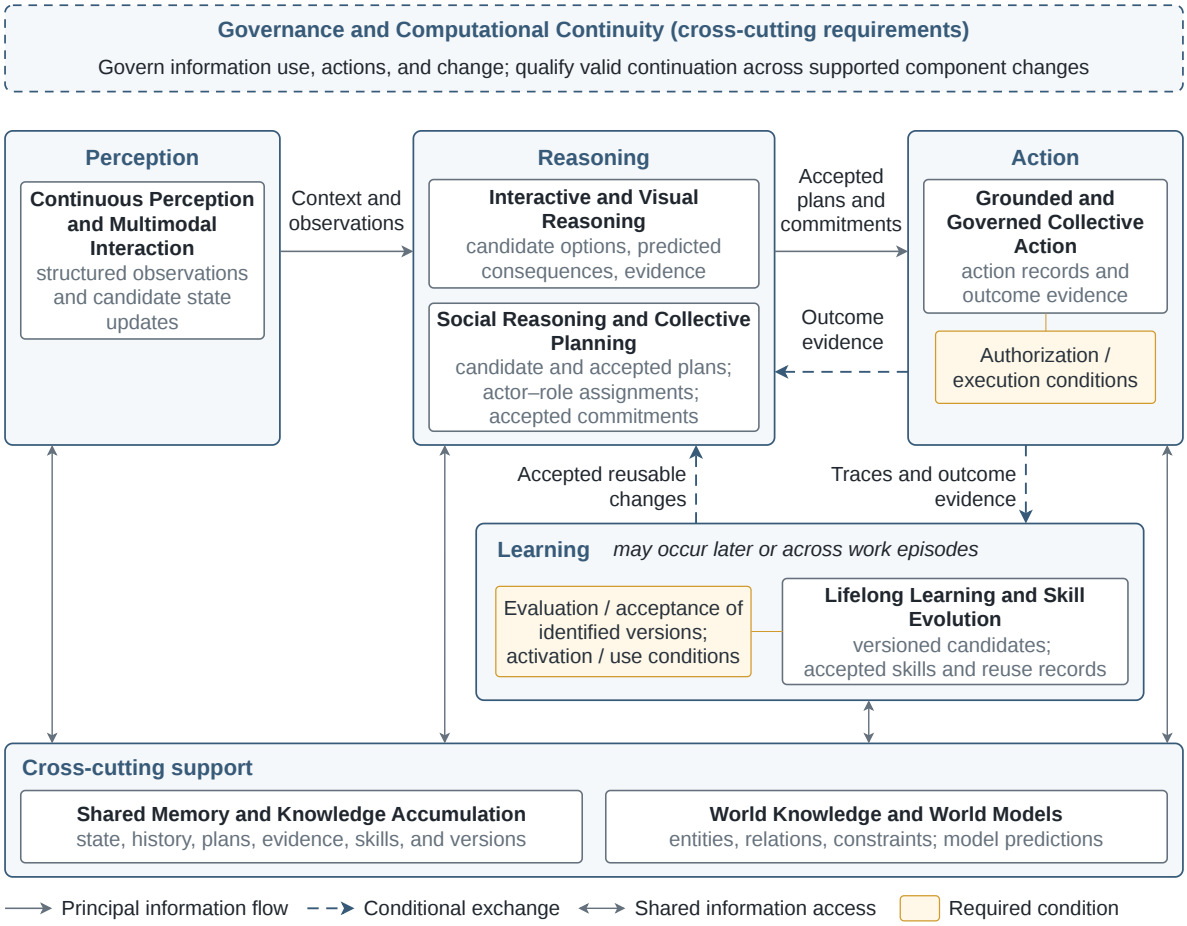



**Figure 2.** Functional relations among the seven capability families. Solid arrows show principal information relations, dashed arrows conditional feedback and exchange, and bidirectional links information access through configured interfaces. Orange boxes identify conditions governing the associated functions. Governance and Computational Continuity apply across these relations. Capability groups can share components and support multiple operational actors.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



### 4.2. Individual Agent Organization

Figure 3 separates an agent's assigned role from the model, harness, and runtime supporting its operation. A role assignment identifies the responsible actor, expected contribution, decision scope, obligations, and period of applicability. Several assignments may belong to one actor or be transferred between actors. Actor boundaries follow Section 3.1.

The foundation model supports interpretation, reasoning, and proposal generation. The agent harness connects model invocation to context, memory, tools, validation, execution control, and tracing. The runtime supplies the execution environment for these functions. CoALA provides an antecedent for organizing model capabilities within memory, action, and decision mechanisms [2]. Here, the model–harness–runtime distinction allows several agents to share a model endpoint or infrastructure while retaining distinct assignments, contexts, permissions, and histories.

An agent operates on an authorized view of relevant state and knowledge. Its proposals enter the appropriate acceptance interface, which may be implemented by its harness, another authorized actor, a shared service, or a human decision-maker. Local context can remain bounded provided the evidence and conditions required for each decision are accessible. Updates affecting shared commitments follow the collective rules below.

Assignment history and configuration metadata link individual operation to the process record. Current state identifies who is responsible now; action and decision records preserve the actor and configuration applicable when each event occurred. Agent replacement, role reassignment, and component replacement are recorded separately. Their effects on ongoing work are assessed through the continuation conditions in Section 5.3.

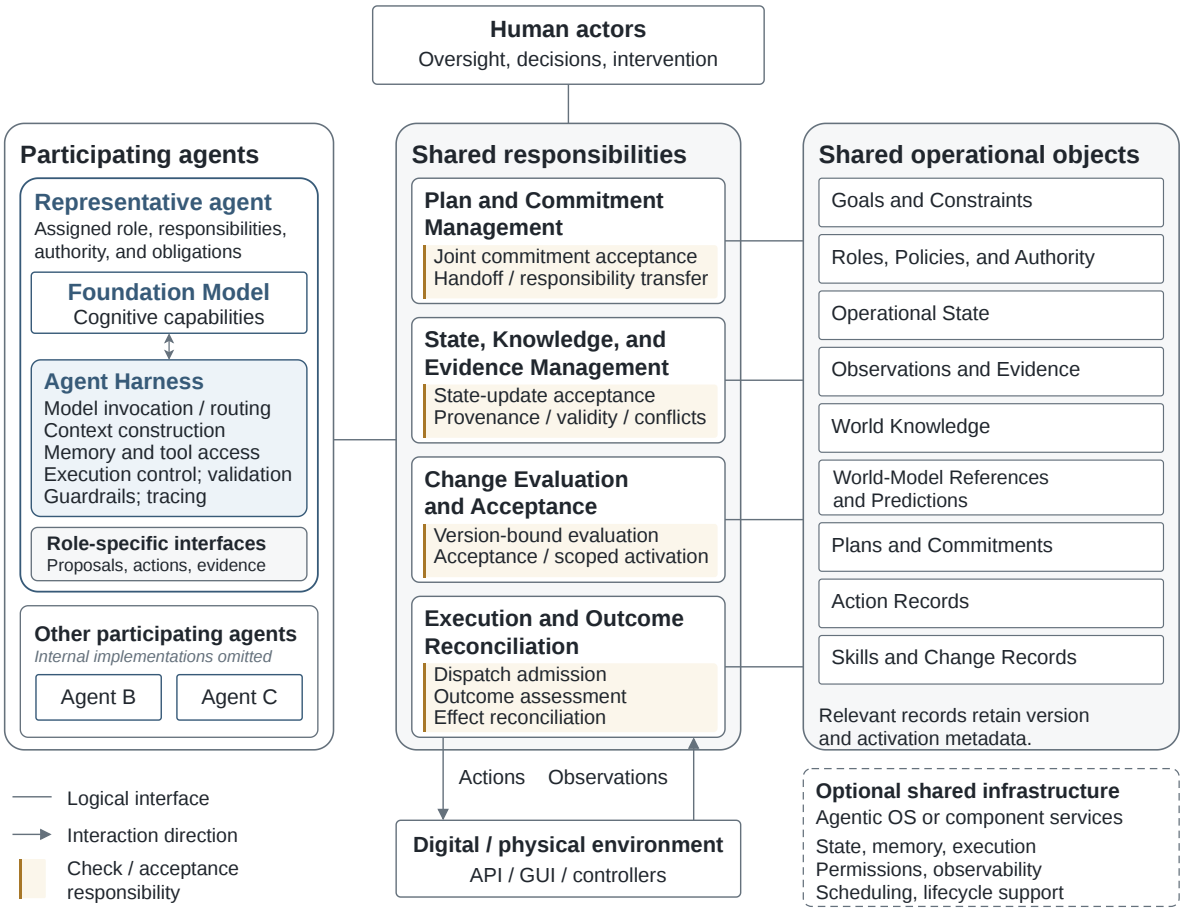



**Figure 3.** Reference architecture relating participating agents, shared responsibilities, operational objects, human actors, and the environment. The expanded agent separates assigned roles, model capabilities, harness functions, and operational interfaces. Connections identify logical interfaces whose implementation may be combined or distributed. Shared infrastructure may use an Agentic OS or a composition of services.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



### 4.3. Shared Responsibilities and Operational Objects

Five responsibility groups connect participating actors to the same work.

*Agent Roles and Contribution Scopes* assign responsibility for information, proposals, decisions, and actions, including which objects an actor may access or modify. An executor is the component or actor performing an execution step; its status as a separate actor depends on its responsibility assignment.

*Plan and Commitment Management* maintains dependencies, resource allocations, assignments, and accepted undertakings. Revisions account for commitments retained, changed, or released.

*State, Knowledge, and Evidence Management* maintains the accepted account of the work and the information supporting it, with provenance, temporal context, validity, and confirmation status.

*Execution and Outcome Reconciliation* connects authorized requests to execution and assessed effects, retaining discrepancies and unresolved outcomes for subsequent decisions.

*Change Evaluation and Acceptance* evaluates identified changes and records their acceptance and activation scope, including affected work and configuration dependencies.

The first group appears within the participating-agent view in Figure 3; the remaining four appear as shared responsibilities. These groups can span several actors or be combined within one actor's scope. A concrete allocation identifies who produces, manages, validates, accepts, and uses the information in Table 3.

**Table 3.** Operational objects connecting responsibilities.

| Object | Information and operational use |
|---|---|
| Goals and Constraints | Process reference, accepted goals, completion conditions, limits, and authorized amendments; predecessor or successor links where applicable. These records identify the undertaking and its accepted terms. |
| Roles, Policies, and Authority | Actor identities, effective assignments, responsibilities, permissions, obligations, approval conditions, and authority sources. These records attribute responsibility and bound decisions and actions. |
| Operational State | Progress, remaining work, resources, uncertainty, current assignments, execution components, configuration references, and activation or suspension status: the current accepted basis for organizing work. |
| Observations and Evidence | Content, provenance, timestamps, affected objects, and confirmation status supporting state updates, monitoring, and outcome assessment. |
| World Knowledge | Domain and environment knowledge with provenance and validity scope, used in interpretation and reasoning. |
| World-Model References and Predictions | Model versions, inputs, assumptions, predicted consequences, and uncertainty, where predictions inform decisions. |
| Plans and Commitments | Dependencies, assignments, allocations, accepted undertakings, versions, and execution conditions organizing collective work. |
| Action Records | Request and work identities, responsible actor, plan, authority, execution configuration, dispatch status, assessed outcomes, unresolved effects, and request-correlated evidence. |
| Skills and Change Records | Candidate, accepted, and active versions; supporting experience, evaluation and acceptance decisions, applicability, dependencies, activation scope, version-linked use and outcomes, and subsequent revision or withdrawal. |

These are logical information structures that may be linked across stores or authoritative external sources. Agent, model, harness, and runtime versions belong to the relevant records or a referenced configuration. Process references and accepted terms remain connected to execution lineage under the identity rules in Section 3.1.

Record management and acceptance authority have separate assignments. Storing a proposal makes it available for assessment; the applicable role, policy, or delegation determines who may accept its operational status. Routine acceptance can be automated within a declared scope. A qualified state claim may remain uncertain, and a prediction retains its assumptions when used alongside observed evidence.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



### *4.4. Interfaces and Responsibility Boundaries*

An interface contract specifies how a contribution may acquire operational status. Building on explicit obligations and conditions at software boundaries [49], each contract identifies:

1. the producer or requester, affected work and objects, and producing configuration;
2. the claim established by acceptance, its qualifications, and permitted downstream use;
3. the decision authority, delegated mechanism, and rule for combining any required approvals;
4. the state, authority, versions, commitments, and environmental conditions governing assessment and use;
5. the evidence and references linking the decision to its inputs, authority, dependencies, and disposition; and
6. the handling of conflict, missing evidence, and uncertainty, including responsibility for follow-up.

Contracts may use executable checks, coordination protocols, or authorized human judgment. An applicable version can remain in use under an explicit compatibility rule; selecting the latest version is a separate deployment choice. Table 4 identifies the principal transition families.

**Table 4.** Interface transitions and their acceptance responsibilities.

| Transition family | Decision responsibility | Conditions and resulting status |
|---|---|---|
| Observation → Accepted State Update | Authority for the relevant state claim. | Establish source, temporal scope, uncertainty, affected state, and valid dependencies. Retain qualifications needed by consumers. |
| Candidate Plan → Committed Plan | Coordination and authorities for affected commitments. | Combine approvals for the same proposal version and compatible dependencies. Establish joint feasibility, authority, assignments, and resource conditions; protect shared constraints when commitments take effect. |
| Action Request → Admissible Dispatch; Outcome Evidence → Outcome Assessment | Execution responsibility and identified outcome-verification responsibility. | Bind dispatch to current work, plan, authority, and execution conditions. Assess request-correlated evidence separately; retain partial or uncertain effects and follow-up responsibility. |
| Interaction Trace → Candidate Skill | Experience-analysis responsibility. | Identify the candidate version, source context, outcomes, assumptions, applicability, and limitations. The resulting candidate enters evaluation. |
| Proposed Change → Accepted Version; Activation Decision → Active Version | Change authority and activation responsibility. | Bind evaluation and acceptance to a version. At activation, establish applicability, compatibility, and affected-work conditions; record actual consumers and pending or partial activation. |
| Handoff Request → Accepted Responsibility and Admissible Resumption | Handoff coordination, receiving actor, and continuation authority. | Identify transferred and retained obligations, reconcile state and uncertain effects, establish compatibility, and resolve overlapping execution rights before dependent work resumes. |

A family may contain several decisions and external operations. The rows identify changes in operational status; their composition follows Section 4.4.1.

#### 4.4.1. Composition Across Acceptance Boundaries

A receiving activity uses an upstream result according to the claim established there, its scope, and its qualifications. It identifies the proposition needed for its own transition and the conditions that remain to be established. An uncertain observation can support investigation, for example, while a resource commitment requires an allocation decision.

For a receiving boundary $b$, the required conditions are summarized by the guard

$$G_b = \mathsf{Match}_b \wedge \mathsf{Basis}_b \wedge \mathsf{Authority}_b \wedge \mathsf{Joint}_b \wedge \mathsf{Use}_b. \tag{2}$$

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



Here, $\mathsf{Match}_b$ requires incoming claims to support the intended interpretation, including uncertainty and confirmation status. $\mathsf{Basis}_b$ requires applicable dependency versions, assumptions, and validity conditions. $\mathsf{Authority}_b$ requires authority for the receiving transition itself. $\mathsf{Joint}_b$ requires combined contributions and affected commitments to satisfy shared constraints. $\mathsf{Use}_b$ requires conditions exposed to change after assessment to remain valid, or to be enforced again, when the transition takes effect.

The decision basis retains work and object references, upstream decisions, relevant versions, authority, evidence, assumptions, and the point at which the new status becomes operative. Equation 2 specifies a contract guard. Its enforcement and any resulting system guarantees depend on the implementation and environmental assumptions.

Where several approvals are required, the governing policy specifies their combination, required authority holders, and treatment of rejection or unresolved conditions. Approvals must concern the same proposed undertaking or change with compatible dependencies. Combining approvals for different versions requires reassessment of their applicability to the accepted version. One actor may perform several decision functions within its assignments while preserving their distinct meanings.

Concurrent acceptance must also protect shared constraints. Two allocations can each pass a feasibility check against the same available resource and still conflict when both take effect. Validation must therefore connect to commitment at the affected resource or state boundary, for example through a conditional update, an enforced reservation, or serialized admission. Checking separate proposal versions covers their individual records; the shared constraint requires coordination over the affected resource and interval.

The same requirement applies between acceptance and later use. A fresh read establishes the observed state at that point; subsequent conflicts require preservation, reservation, or a check linked to effective admission. An external interface bounds the effect that local acceptance can establish. Monitoring supports prevention when connected to such admission control, and otherwise supports detection and response.

Each consumer of an accepted result establishes its own authority and remaining prerequisites. Shared use of a record remains bounded by any limited allocation or execution right it represents. An upstream change triggers reassessment of dependent uses whose premises are affected. Earlier decisions remain in the history, with explicit disposition of existing commitments and external effects.

For transitions spanning several updates or external operations, the contract records completed, pending, failed, and uncertain parts against declared completion conditions. Downstream work proceeds only within the scope supported by that combination. Partial activation can be accepted for identified consumers under compatible conditions; completion for a wider scope requires evidence covering that scope. Later failure leaves earlier effects and obligations to be reconciled, revised, or addressed by authorized compensation.

#### 4.4.2. Dispositions and Reassessment

The assessment returns *accept*, *reject*, *wait for evidence*, or *escalate*. Acceptance permits the specified transition; rejection records why the submitted transition is inadmissible. Waiting identifies missing evidence and a responsible owner. Escalation refers the issue to an appropriate decision-maker while preserving its pending acceptance status.

Deferred, escalated, or cyclic dependencies require a reassessment trigger and any applicable deadline or termination rule. Reassessment checks current state, authority, and versions. A timeout triggers this handling rather than satisfying an unmet condition. Independently admissible work may continue while affected transitions remain pending. Uncertain execution outcomes require the reconciliation described in Section 5.3.

### *4.5. Execution, Coordination, and Adaptation Loops*

The lifecycle follows three interacting loops (Figure 4). Autonomic computing provides precedents for component interactions and feedback management [17,18]. Here, the loops are connected through the operational objects and acceptance contracts of the ongoing work.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



The *execution loop* selects the next admissible step, establishes dispatch conditions, and assesses resulting evidence. Local choices remain within accepted commitments and delegated authority. Changed shared obligations or invalidated assumptions trigger coordination.

The *coordination loop* develops proposals, resolves dependencies and resource conflicts, and accepts or revises commitments. Replanning can repair current work while leaving reusable behavior unchanged.

The *adaptation loop* analyzes contextualized experience, evaluates candidate knowledge, skills, workflows, or configuration changes, and records acceptance, activation, reuse, and subsequent feedback. Evidence for improvement comes from evaluation of the resulting behavior.

The loops may interleave, operate at different time scales, or remain inactive until relevant conditions arise. Execution evidence can trigger replanning or experience analysis; accepted changes can affect planning or execution after applicability and activation checks. A plan revision can invalidate pending dispatch, a policy change can affect activation, and an unresolved action can constrain both replanning and handoff. Receiving activities apply Equation 2 to these dependencies.

Changes in participating agents are handled through the same loops and continuation contracts. Computational Continuity spans their affected responsibilities. The property-to-architecture crosswalk appears in Appendix B.3.

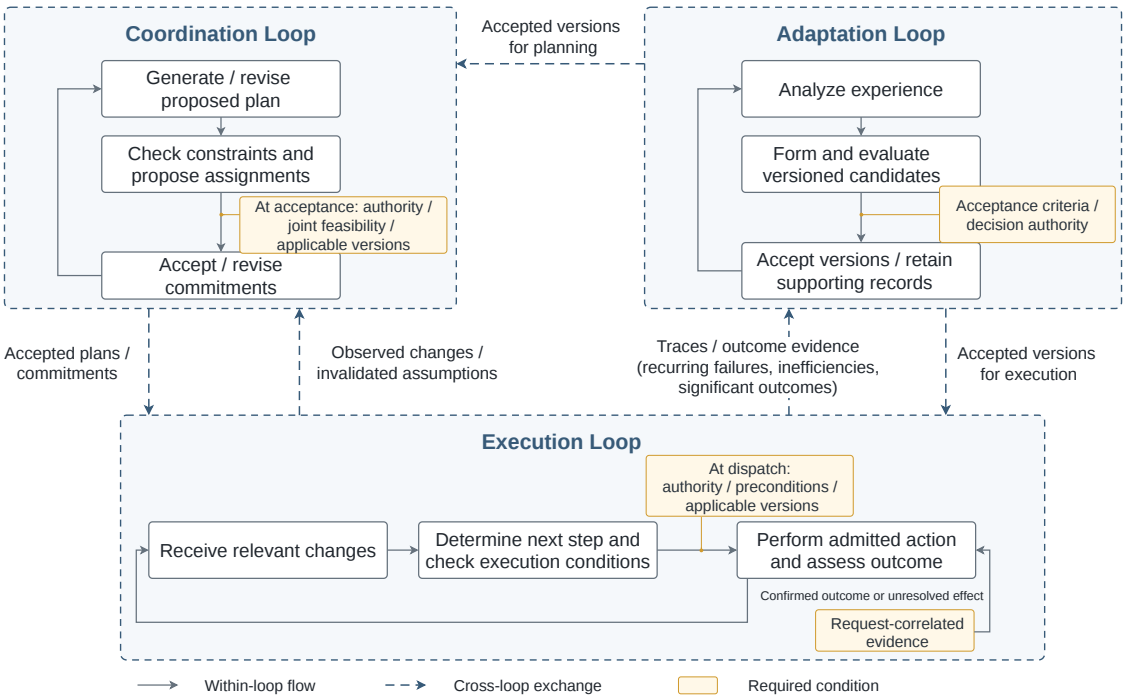



**Figure 4.** Execution, coordination, and adaptation loops. Dashed arrows indicate conditional exchanges of plans, observations, outcome evidence, and accepted changes. Highlighted annotations identify acceptance and use conditions. Receiving activities apply the composition guard in Section 4.4.1; accepted changes additionally require activation in the receiving context.

### *4.6. Deployment and Resource Conditions*

A deployment assigns the logical responsibilities to agents, harnesses, runtimes, shared services, external systems, and human decision-makers. For each transition, it identifies the mechanism provider, authoritative information source, acceptance authority, and owner of unresolved conditions.

Consistency and availability requirements follow the decision being supported. Historical analysis may tolerate delayed records; commitment acceptance must protect affected resource constraints, and recovery requires evidence about external effects. Resource budgets cover execution, retrieval,

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



verification, and evidence retention. When a required condition cannot be established within those budgets, the affected work is deferred or its scope is revised under applicable authority.

Appendix B.2 develops placement choices, Agentic OS support, disconnected operation, and decision-specific consistency requirements. These choices preserve the contracts above while varying how their mechanisms are deployed.

## 5. Governance, Assurance, and Computational Continuity

The contracts in Section 4 locate decisions about operational status. This section develops their authority, evidence, and continuation requirements, including changes affecting work already in progress.

### *5.1. Roles, Norms, Protocols, and Authority*

Governance assigns decision rights, execution authority, and obligations. Electronic institutions connect roles to interaction conventions [14]; role-based access control associates permissions and constraints with role assignments [50]. A configuration specifies each assignment's scope, validity, revocation conditions, and escalation path. Least privilege and separation of privilege inform its technical enforcement [51].

Proposal, agreement, authority, and authorized action retain distinct meanings. Agreement establishes authority where an existing rule grants that effect to the participants. Execution also requires technical permissions, current preconditions, and compatibility with commitments. The configuration connects an enforcement mechanism to its authority source and identifies who may amend that basis.

AgentSpec specifies runtime rules through triggers, predicates, and enforcement responses [52], while GuardAgent generates and executes guardrail code against proposed actions [53]. Policy-following evaluation in $\tau$-bench and tool-output attacks in AgentDojo examine related enforcement and reliability problems [26,27]. Their use within this architecture requires an authorized policy basis and a defined enforcement scope.

### *5.2. Runtime Monitoring and Outcome Evidence*

Assurance combines pre-action validation, runtime monitoring, outcome confirmation, process compliance checking, and post-event assessment. Runtime verification examines observed execution against specified properties [54]; process conformance analysis compares recorded behavior with required procedures [21]. Pre-action checks use the admission conditions in Section 4.4.1, including protection against relevant changes between assessment and use.

Action records distinguish proposal, authorization, dispatch, processing, and confirmed success, confirmed failure, or an uncertain outcome. Cancellation, authority expiry, and late evidence are recorded against the request while preserving unresolved effects. Confirmation can also establish a specified partial effect. Its meaning depends on the evidence required for that action: request receipt, application completion, and physical completion establish different conclusions. The end-to-end argument explains the need for application-level evidence beyond delivery acknowledgments [55].

Outcome assessment uses request-correlated API responses, authoritative records, sensor observations, human confirmation, or execution histories, with source, time, and configuration context. Provenance supports interpretation and diagnosis [56]; agent identifiers, monitoring, and activity logs support attribution and inspection [57]. Partial observations can leave the required conclusion unresolved. Predictions and agent self-assessments can guide further checks, while confirmation of external effects requires evidence traceable to those effects.

MAST identifies premature termination and incorrect or incomplete verification among multi-agent failure patterns [41]. Each assurance claim therefore states the requirement, mechanism, evidence, supported conclusion, and limitation. Repeated-trial reliability and evaluation cost qualify that claim [26,28]. Appendix B.4 distinguishes these claims from intrinsic self-correction and formal verification results.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



### 5.3. Continuity Across Agent, Model, Runtime, and Shared Infrastructure Changes

A continuation decision concerns identified work under its current accepted terms. Process identity and execution lineage follow Section 3.1; resumption requires evidence that the receiving configuration can assume the outstanding work under current conditions.

Rollback-recovery protocols address consistent recovery, logged events, and interaction with external outputs [10]. An agent checkpoint can still precede an effect whose completion record was lost. Likewise, a missing RPC reply leaves execution uncertain [58]. Recovery must reconcile that uncertainty before a retry or establish applicable idempotency or duplicate-detection guarantees. Evidence about external effects and control of future execution rights address separate obligations.

Within the declared scope, accepting handoff or recovery requires:

1. identifiable process reference, accepted goals and completion conditions, authorized amendments, current state, active configurations, responsible actors, and authoritative records;
2. separation of completed, in-progress, not-yet-started, and issued-but-uncertain actions, with reconciliation or applicable duplicate-handling guarantees for retries and evidence of prerequisites for dependent work;
3. compatibility of the receiving configuration with required state, interfaces, resources, and operational conditions;
4. confirmed or re-established continuation authority, with the previous executor's ability to produce overlapping effects revoked, restricted, or otherwise resolved; and
5. a record of transferred and retained responsibilities, residual uncertainty, and conditions for resumed work.

Recipient-checked sequencers in Chubby provide a precedent for execution-right control [19]. Such controls address which executor may act; outcome reconciliation establishes what earlier requests already produced. Both remain relevant when a handoff is initiated because the previous executor is unavailable.

An infrastructure restart follows the same conditions. Restored records are reconciled with authoritative external information, existing commitments, and any remaining ability of the previous executor to act. Expired authority, superseded commitments, and obsolete configuration state are updated or excluded from the resumption basis. If a required condition remains unresolved, affected work waits or follows an authorized escalation or recovery procedure. Continuity therefore permits reconciliation pauses and applies to the supported changes declared for the configuration.

### 5.4. Governed Adaptation and Versioned Change

The adaptation loop produces candidate changes with supporting experience and an intended applicability scope. Evaluation examines an identified version against declared acceptance conditions; acceptance records the version, evidence, scope, and adopting authority. Activation establishes where that version becomes operative, using current dependencies and authority under Equation 2.

Evaluation addresses the changed object and its consumers. Documentation of intended uses and exclusions [59], retention, transfer, and interference [22], and configuration dependencies [60] inform this assessment. The change record identifies the proposer, evaluation and acceptance decisions, target consumers, activation conditions, and any migration, rollback, or compensation procedure. Changes to the evaluated candidate or its relevant conditions require reassessment.

Requested activation and established activation are recorded separately. Partial adoption identifies actual consumers, pending consumers, and ongoing work remaining on a supported earlier version. Mixed versions require compatible coexistence conditions. Later retrieval makes a retained version available for consideration; application requires compatibility with the receiving work. Its version, use context, and resulting outcomes are recorded for subsequent evaluation, revision, withdrawal, or deprecation.

Changes to goals, constraints, or completion conditions follow the identity rule in Section 3.1. An amendment records revised terms, authority, effective point, and the disposition of affected obligations

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



and completed or uncertain actions. A successor undertaking receives a distinct reference and explicit treatment of inherited or retained obligations. Earlier outcome records continue to describe what was established under the terms applicable at the time.

Policy changes require acceptance under authority valid before activation. The candidate policy acquires force through that decision, preserving an independent basis for its adoption. Affected permissions, obligations, and commitments are then reviewed under the accepted policy.

Withdrawal or rollback also accounts for effects already produced. Restoring an internal version requires authority and compatibility with the remaining work; external effects require reconciliation and, where appropriate, separately authorized compensation. Sagas provide a precedent for compensating partially completed activities [61]. Compensation has its own preconditions, outcome evidence, and execution record. Idempotency, reversibility, and compensability remain distinct action properties.

Table A6 in Appendix B.4 summarizes the affected-work checks by change type. Section 6 applies the contracts and continuation conditions to the constructed trace, counterexamples, and configuration profiles.

## 6. Architectural Analysis and Integrated Worked Example

We examine the framework through a constructed production-order trace, controlled changes to its decision conditions, and implementation-source profiles of selected configurations. The trace specifies witnesses that would justify each transition; the profiles assess relationships visible in pinned source paths. These evidence bases support different conclusions. The analysis concerns decision requirements and scoped architectural findings, with implementation reliability and comparative effectiveness left for separate evaluation.

### *6.1. Scenario and Assessment Scope*

Production order $O_1$ requires preparation, processing, and quality control under an accepted deadline, budget, and quality conditions. Its initial plan $P_1$ uses machine $M_1$. Alternative $M_2$ requires checks of availability, scheduling, and operating conditions; technically capable $M_3$ lies outside the current authority scope. Plan revisions preserve $O_1$'s accepted terms in the principal branch.

Four distinct agent actors carry interdependent responsibilities: Production Planning develops plans and allocations; Inventory / Procurement establishes material conditions; Maintenance assesses machines and usage restrictions; and Execution performs authorized actions and supplies outcome evidence. A Human Supervisor handles decisions within its assigned authority, including designated escalations. Shared models or infrastructure leave these responsibility boundaries intact.

The environment combines APIs and a legacy GUI. In the handoff, $E_1$ and $E_2$ are outgoing and incoming execution instances serving the same Execution actor. The instance change retains that actor's responsibility; $O_2$ is a separate undertaking. The scenario uses one organizational authority context, obtainable but potentially delayed or conflicting evidence, bounded predictive representations, and explicitly introduced disruption branches. Authority and record sources serve as the reference basis; their compromise is outside these branches.

Appendix C records the assumptions, checking conditions C1–C7, and complete transition ledgers. Witness references specify required claims, sources, versions, temporal scope, and qualifications. Their satisfaction is stipulated along the selected successful $O_1$ branch. T11–T12 instead retain unresolved evaluation and reuse requirements. The trace is a specification-level construction under the claim-basis distinction in Section 3.3.

### *6.2. Integrated Operational Trace*

Figure 5 follows the work through commitment, execution, disruption, continuation, and candidate formation. Step identifiers describe dependencies; independent activities may overlap. Tables A7 and A8 in Appendix C retain each step's witnesses, decision responsibility, resulting status, and obligations.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



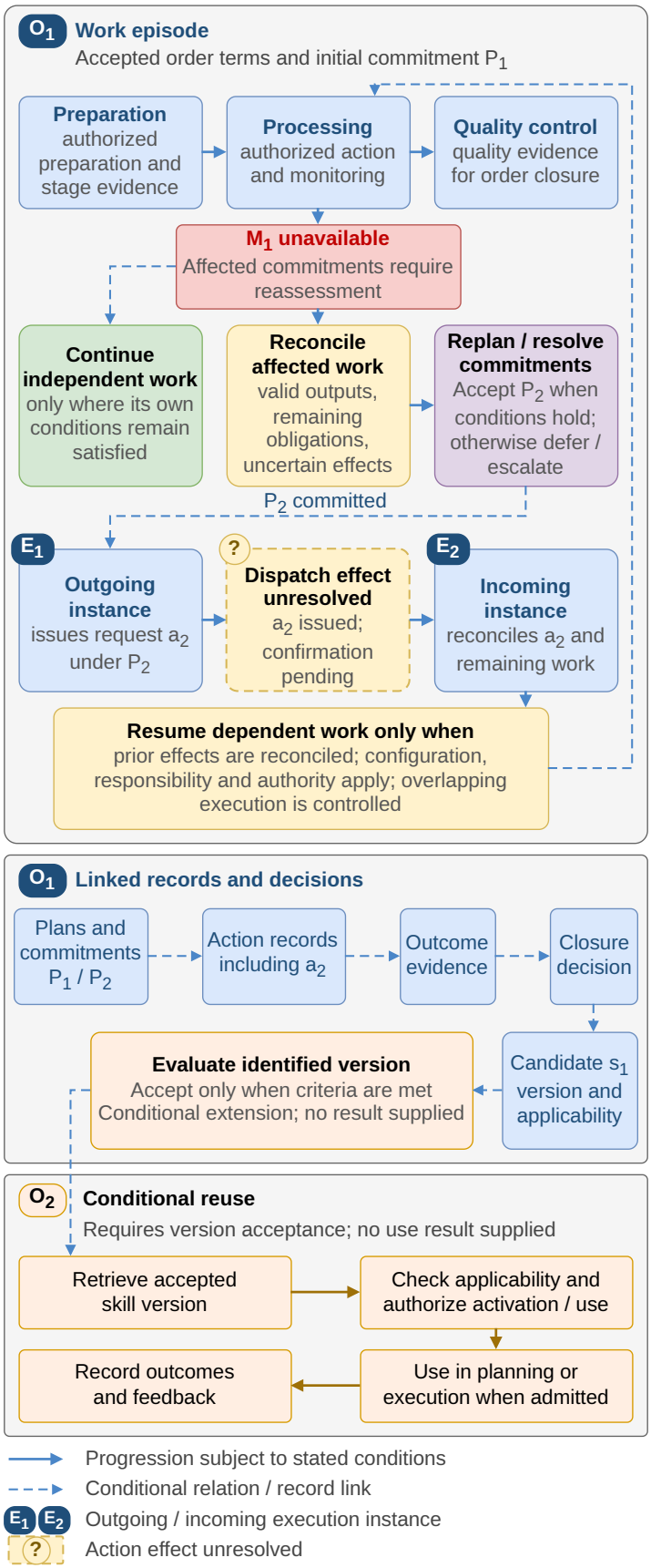



**Figure 5.** Constructed trace for $O_1$ and conditional experience reuse in $O_2$. Resource disruption requires reconciliation and commitment revision. Resumption across $E_1$ and $E_2$ requires compatible reconciliation and handoff results. T11–T12 specify evaluation and reuse conditions whose results remain unresolved.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



6.2.1. Commitment, Execution, and Resource Revision

T00–T03 establish the accepted order terms, qualified operational state, and a candidate plan. Resource observations and predictive estimates retain distinct statuses. At T04, $W_{P_1}$ binds commitment to the same proposal version, applicable authority, and resource conditions protected when allocation takes effect. T05 confirms preparation completion and the start of processing on $M_1$; processing completion and quality control remain outstanding.

When $M_1$ fails at T06, Execution reconciles progress and identifies outputs requiring revalidation. Production Planning, Inventory / Procurement, and Maintenance assess $M_2$ against materials, scheduling, downstream dependencies, quality, deadline, budget, and existing commitments. In the selected branch, $W_{\text{revision}}$ supports $P_2$, with affected commitments retained, revised, or released. Figure 6 shows this decision. Unresolved resource conditions defer the affected commitment; use of $M_3$ additionally requires an applicable authority change.

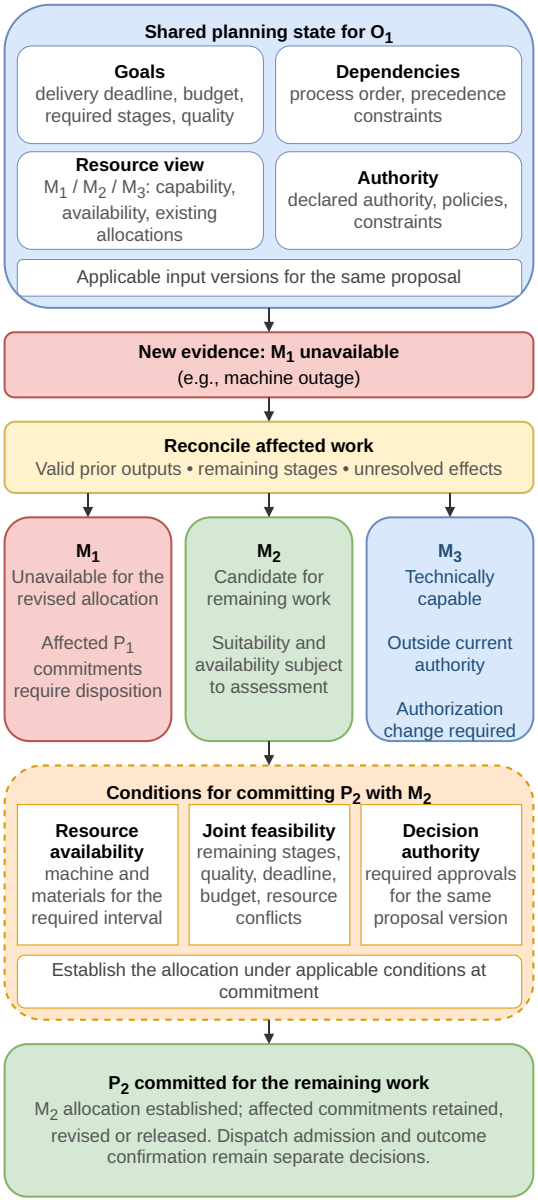



**Figure 6.** Shared planning representation at T06. Evidence of $M_1$'s unavailability prompts reconciliation and assessment of alternatives. Committing $P_2$ with $M_2$ requires applicable versions, resource availability, joint feasibility, authority, and an effective allocation. These conditions are stipulated in the displayed branch. $M_3$ requires an authority change; dispatch and outcome assessment remain separate decisions.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



#### 6.2.2. Uncertain Dispatch and Admissible Resumption

At T07, $E_1$ issues GUI request $a_2$ to dispatch the remaining processing to $M_2$ under $P_2$. Handoff begins before its effect is confirmed. The retained record identifies the request, work, plan, execution configuration, and unresolved status.

T08a uses $W_{\text{dispatch}}$ to correlate an enterprise record with $a_2$, $O_1$, the processing stage, and $M_2$. In the selected branch, this establishes dispatch, leaving processing completion to separate evidence. T08b establishes $W_{\text{handoff}}$: receiving-configuration compatibility, retained actor responsibility, permitted continuation scope, and effective control of $E_1$'s overlapping execution, including treatment of outstanding requests.

T08c combines these results against current $P_2$ dependencies. The work resumes with $E_2$ and $a_2$ remains unrepeated. Reconciliation and handoff preparation may overlap, while dependent effect-producing work requires their compatible completion. Authorized investigation can proceed within its own scope while either condition remains unresolved.

Retry requires particular care. Evidence of non-execution must also establish that the outstanding request can no longer produce the effect. A retry under uncertainty would require an applicable duplicate-suppression guarantee covering request identity, delayed delivery, and the receiving environment. The legacy-GUI branch provides no such guarantee. Revoking future execution rights also leaves already admitted requests subject to reconciliation.

At T09, separate evidence supports the remaining processing and quality-control outcomes. Closure additionally accounts for the deadline, budget, and commitment dispositions. The selected branch completes $O_1$ under its accepted terms; missing quality evidence would retain the corresponding obligation and prevent successful closure.

#### 6.2.3. Candidate Formation and Conditional Reuse

At T10, the actors derive candidate response skill $s_1$ from the $O_1$ trace. Its identified version retains applicability, resource and authority conditions, required evidence, and the distinction between effect reconciliation and admissible resumption.

T11 requires evaluation of that version against declared criteria, including unavailable resources, unresolved effects, and insufficient authority. The example supplies no evaluation result, so $s_1$ remains a candidate. T12 specifies later use in $O_2$, conditional on T11 acceptance, receiving-context evidence $W_{\text{use}}$, and an applicable activation decision. Actual use would link the version to the decisions or actions it informs and their subsequent outcomes. Evaluation, reuse, and improvement therefore remain open empirical questions for this pathway.

### *6.3. Controlled Checks and Counterexamples*

Table A9 changes one decision-relevant condition at a time. K1–K2 examine proposal binding and the validity of prior outputs; K3–K4 separate request-specific reconciliation from execution-right control; K5 checks closure evidence; K6–K7 examine evaluation-version binding and receiving-context applicability; K8 separates feasibility from authority. The stated dispositions concern the first affected transition. They are analytical expectations, with later consequences dependent on any remaining controls.

A resource-allocation counterexample makes the composition problem explicit. Suppose $M_2$ supports one operation during interval $I$, and $c_a, c_b \in \{0, 1\}$ indicate two operative commitments. The shared constraint is $c_a + c_b \leq 1$. Both actors read the same unallocated state and independently pass their local checks. If acceptance updates their separate proposal records without protecting the shared allocation, both can become operative, giving $c_a + c_b = 2$.

The missing relationship connects validation to effective allocation for that resource and interval. A conditional allocation or equivalent controlled admission must preserve the shared constraint; a conflicting proposal requires reassessment, deferral, or rejection. This instantiates $\text{Joint}_b$ and $\text{Use}_b$ in Equation 2, while unrelated work may proceed under its own conditions.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



Appendix C.5 examines the remaining counterexamples: unauthorized agreement, predictions accepted as outcomes, duplicate dispatch, overlapping execution, incomplete continuation summaries, overgeneralized skills, and trace retention without an experience-use path. Each identifies a missing decision relationship and the conditions under which it matters. Together with C1–C7, these cases locate evidence requirements across P1–P6; they provide neither an implementation profile nor a proof over all possible interleavings.

### *6.4. Evidence Profiles of Selected Configurations*

We apply Codebook v1.0 to three application configurations and one infrastructure boundary case. The assessment uses implementation-source evidence (basis I) inspected on September 29, 2026. Appendix D retains the pins, configuration boundaries, source locators, and criterion-level rationales. The assessment is static, with no framework execution or new performance evaluation.

Table 5 reports each property label and its criterion vector. Judgments follow Section 3.3: N requires a configuration-specific negative witness, while unresolved requirements retain U. Evidence from other versions or optional extensions remains outside the selected scope.

**Table 5.** Scoped implementation-source profiles under Codebook v1.0 (basis I). Vectors follow the constituent-criterion order; labels follow Equation 1.

| Configuration | Assessed scope | P1 | P2 | P3 | P4 | P5 | P6 |
|---|---|---|---|---|---|---|---|
| MetaGPT v0.8.2, `df9bc18` [62] | Local software-company workflow with implementation, code review, and tests enabled; ordinary role memory, project artifacts, and serialized-team recovery. Records MG-P1–MG-P6. | PS EUU | PS EUU | PS EUU | PS UUEU | PS EUUU | II UUUU |
| LangGraph 1.2.11, *Interrupts in tools* [63,64] | Email-send task with SQLite checkpointing, interrupt/resume, caller approval, and a print-only email placeholder. No Store, subagents, or learning extension. Records LG-P1–LG-P6. | PS EUU | NS NUU | II UUU | NS UENN | PS EUUU | NS NUUU |
| Voyager, `55e45a8` [65] | Automatic single-bot Minecraft configuration with internal curriculum, action, and critic components, executable skills, checkpoint/resume, and later-task reuse. Records VG-P1–VG-P6. | PS EUU | NS NUU | NS NUU | PS UUEU | PS EUUU | NS EENU |
| AIOS v0.2.2, `57046b5` [66] | Local Python kernel with FIFO scheduling and execution, memory, tool, and file services; context management disabled, with no bound application workflow. Records AI-P1–AI-P6. | II UUU | II UUU | II UUU | II UUUU | II UUUU | II UUUU |

P1–P6 follow Section 3.2. Criterion codes and property labels are defined in Section 3.3. The selected cases illustrate different relationships; the profiles describe evidence within scope rather than a representative sample or performance ranking.

Actor boundaries explain part of the variation. MetaGPT's interdependent role instances support P2.1, whereas Voyager's internal components serve one operational actor. In the LangGraph example, the caller's operational-actor assignment remains unspecified, leaving the cross-actor knowledge relationship unresolved. AIOS supplies infrastructure without a bound application-level assessment unit. Its all-U vectors identify that scope limitation.

### *6.5. Diagnostic Findings*

A diagnostic finding connects evidence to an affected decision, a supported conclusion, and a targeted follow-up check. Table 6 reports six findings. DR1–DR2 concern constructed cases; DR3–DR6 use the source dossier. This distinction separates a stipulated transition violation, a source-path negative witness, and an unresolved assessment requirement.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



**Table 6.** Decision-specific findings and follow-up checks.

| Record | Evidence and finding | Affected decision or follow-up |
|---|---|---|
| DR1: allocation counterexample | Two locally feasible commitments exceed shared capacity when both take effect. Their admission leaves the shared resource constraint unprotected. | Connect acceptance to an effective resource-allocation mechanism; defer or reject conflicting commitments. |
| DR2: T07–T08c; K3–K4 | State restoration can leave $a_2$ unresolved or $E_1$ able to act. Effect reconciliation and execution-right control are separate continuation conditions. | Correlate evidence with $a_2$; establish responsibility, authority, compatible state, and control of overlapping rights and outstanding requests before dependent resumption. |
| DR3: LG-P4 | The approved path prints and returns success without an email-service action. For the represented email effect: P4.1 U, P4.2 E, P4.3–P4.4 N; NS (`UENN`). | Bind the request to an actual email action and evidence for its specified outcome. Establish the approving caller's authority separately. |
| DR4: VG-P6 | Critic-gated skill admission coexists with same-name active replacement lacking acceptance of affected dependent use. P6.3 is N; NS (`EENU`). | Inspect dependency resolution and the acceptance covering affected uses when bindings change. Historical files address retention only. |
| DR5: MG-P5 | Serialized team state establishes P5.1; P5.2–P5.4 remain U. PS (`EUUU`) describes partial evidence for continuation. | Seek the undertaking and obligations, uncertain-effect treatment, receiving compatibility, continuation authority, and execution-right controls. |
| DR6: AI-P1–AI-P6 | Kernel services are specified without a bound application. All-U vectors yield II across P1–P6. | Define the application, actors, authority, operational objects, and supported changes before reassessment. |

DR1–DR2 derive from the constructed cases. DR3–DR6 retain the source basis and judgments in Appendix D. Follow-up checks specify assessment needs; their implementation and effectiveness remain unevaluated here.

DR3 and DR4 identify source-path limitations within the selected configurations, rather than observed operational failures. DR5 and DR6 instead identify evidence or scope requiring clarification; U provides no basis for asserting an implementation defect. In particular, historical skill retention and active dependency binding require separate inspection, as the Voyager witness in Appendix D.3 shows.

The result is a traceable connection between architectural questions and decision-specific findings. Established approaches combined as discussed in Section 2.4 could support related analyses. Comparative diagnostic accuracy, assessor agreement, assessment cost, and relationships to operational performance require the validation described in Section 7.6.

## 7. Discussion and Research Agenda

The analysis in Section 6 connects architectural relationships to specific decisions and evidence requirements. This section considers their implications across domains and the studies needed to assess the framework's diagnostic and operational value.

### *7.1. Scope, Limitations, and Classification Validity*

Agentic AI 2.0 provides a reference organization of established concepts for ongoing foundation-model-based work. Its contribution lies in connecting operational responsibilities, contracts, and criterion-level evidence within one analysis. The predecessor comparison in Section 2.4 identifies established approaches that can realize these relationships. The six-property synthesis remains revisable, consistent with conceptual-framework methodology [67]. Its minimality, exhaustiveness, and incremental diagnostic value remain open questions.

Profiles describe interdependent responsibilities within a declared configuration and workload. Equivalent operational relationships should receive equivalent judgments across software packaging choices; changes in actor roles, process identity rules, authority, environmental assumptions, or the changes being assessed may justify reassessment. Shared mechanisms can support several properties, with evidence required for each relationship. The resulting labels describe evidential situations rather than an additive maturity scale. Equation 1 fixes aggregation once criterion judgments are assigned.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



Reliability therefore depends on evidence admissibility, criterion completeness, and the applicability of negative witnesses. Documentation quality and source accessibility may influence those judgments.

The acceptance guard in Section 4.4.1 specifies how local decisions connect. Its implementation requires coverage of the relevant dependencies, authoritative version and resource information, effective admission control, and the stated behavior of external interfaces. Global consistency and progress under concurrent change and failure require separate implementation-level assurance, including observation of effects, outstanding obligations, and reassessment across connected boundaries.

The current evidence comprises constructed analyses and static source inspection. The production trace stipulates witnesses for its successful branch, while candidate evaluation and later cross-work reuse remain conditional. The three application configurations and the AIOS infrastructure boundary case provide traceable source findings within selected scopes. Their small, purposive selection limits generalization; independent assessor agreement and operational performance remain unmeasured.

The framework's targeted source selection and retrospective construction also leave literature saturation and independent reproducibility of the synthesis open. Alternative splits and merges should be compared with configuration evidence and relationship-level judgments held fixed, following the broader concern with classification usefulness in taxonomy development [68]. Relevant outcomes include explanatory usefulness, assessor agreement, stability, and identification of consequential gaps. Comparisons should account for decomposition granularity and separate changes in the system from changes in definitions, grouping, or evidence.

### *7.2. Domain Implications*

The prospective profiles in Appendix E apply the framework to enterprise productivity, adaptive manufacturing, and distributed mobility. They specify design and evaluation requirements; property-level support judgments require evidence from an assessed implementation.

Enterprise work places particular weight on accepted decisions, pending approvals, and responsibility transfer across business applications and human roles. The practical question is whether a case can progress or change owner with its obligations and decision authority intact. Manufacturing couples these obligations to physical resources, precedence, and quality conditions. A software rollback leaves physical effects to be reconciled, and rescheduling must respect work already performed and commitments to other orders. Mobility adds observation age, reachability, spatial constraints, and locally bounded authority. A delayed assignment may arrive after its original decision basis has changed, making communication conditions part of its admissibility.

These differences determine what counts as adequate evidence and which costs matter. Enterprise evaluation emphasizes rework, unresolved approvals, and supervisory effort; manufacturing adds physical outcome verification and quality; mobility emphasizes state freshness and commitment preservation during reassignment or disconnection. Table A14 develops these requirements under the continuation and reuse rules established in Section 5.

### *7.3. Coordination and Resource Constraints*

Task allocation, distributed constraint optimization, and multi-agent path finding provide established treatments of assignment and joint feasibility [69–71]. The unresolved design question for ongoing work is how to maintain the affected commitment and resource dependencies as participants, capacity, authority, and information change. In particular, which dependencies must be represented to identify the smallest coordination boundary within which a disruption can be resolved?

Local repair is attractive when most work remains valid, but a revised commitment can affect allocations beyond the immediate task. Central coordination can simplify consistency while concentrating communication, computation, and failure risk. Local decisions can be timely while relying on stale or incompatible views. The choice of centralized, distributed, or hybrid control therefore depends on dependency structure, communication constraints, action reversibility, and the cost of conflicting commitments. Richer representations may expose more conflicts while increasing synchronization and maintenance costs.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



Evaluation should vary contention, partial information, reassignment, and disruption, recording joint feasibility, commitment violations, resource conflicts, reassignment latency, and coordination overhead. It should also examine recovery from stale allocations and, under sustained contention, deadlock, starvation, and the distribution of service. These measures can reveal coordination failures hidden by eventual task completion and identify when additional agents improve useful parallelism relative to their coordination cost.

### *7.4. Memory, Prediction, and Grounding Uncertainty*

Memory architectures can retain observations and abstractions for later behavior [72]. Ongoing work adds a retention question: which information must survive retrieval, compression, and transfer for a later decision to remain justified? Relevant items include provenance, temporal validity, superseded versions, unresolved source disagreement, provisional decisions, and responsibility for outstanding obligations. Evaluation should measure decision-relevant preservation alongside retrieval usefulness and context cost.

Partially observable decision models formalize action under uncertain state [73]. In the present setting, stale observations, conflicting sources, uncertain inferences, forecasts, and unconfirmed action effects call for different responses. Semantic uncertainty estimates can help detect unsupported model answers [74]; their operational use still requires identifying the proposition and consequences at issue. High confidence may accompany outdated information, while a reversible exploratory action may be reasonable under uncertainty. A single confidence value would obscure these differences.

Grounding proposals in available skills and affordances addresses action feasibility [75]. Permission and evidence of the resulting effect require their own checks. The research problem is to preserve these distinctions at manageable cost as information passes through predictions, summaries, plans, and handoffs. When evidence contradicts a forecast or supersedes a record, the system must identify the decisions and commitments that depend on it.

Longitudinal tests should track freshness, provenance loss, operational use of superseded information, calibration, and the effect of prediction errors on commitments. They should also measure unsupported state updates, outcome-confirmation errors, retries under uncertain effects, and appropriate evidence seeking, replanning, or escalation. Retention and synchronization costs belong in the same evaluation.

### *7.5. Governance and Adaptation in Open Environments*

Distributed norm management addresses changing obligations, permissions, and prohibitions in open multi-agent settings [76]. For ongoing work, a central question is how revised authority or policy affects commitments already accepted. Delegation can expire, be revoked, or transfer across actors, assets, and jurisdictions. Resolution rules must specify whether affected work retains its earlier basis, undergoes reassessment, or waits for an authorized decision. Conflicts between an existing obligation and a new prohibition also require propagation to dependent work.

Adaptation raises a related question about the evidence needed to introduce change. Continual learning exposes interference between new experience and existing capabilities [22]; changes to prompts, skills, planners, routing, or models can likewise alter approval and action behavior outside the motivating case. Building on Section 5.4, research should relate evaluation and activation requirements to impact, reversibility, and applicability. Migration during active work must balance compatibility with timely access to fixes, with explicit conditions for version coexistence, handoff, and recovery.

Runtime assurance provides monitoring and reversion mechanisms for adaptive components [77,78]. Extending this approach to agentic work requires attention to organizational authority, commitments, and reuse conditions alongside physical or technical operating bounds. Human review should supply a specified decision right or information need, with its delay and workload measured.

Evaluation should examine authority-revocation latency, propagation of policy changes, unauthorized actions, and detection of normative conflicts before execution. Change studies should test regression, applicability errors, effects on unrelated workflows, and recovery or rollback in the presence

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



of external effects. The responsible configuration, applicable authority, and version-linked outcomes should remain traceable through these changes.

### *7.6. Evaluation, Empirical Validation, and System Realization*

A validation program should distinguish the consistency of the assessment procedure, its diagnostic utility, and the operational effects of implementing the proposed relationships. This follows the design-science distinction between constructing an artifact and establishing its utility in use [79].

Assessment reliability and diagnostic utility.

Independent coding studies can assess the reliability of the procedure [80]. Assessors should use the same codebook version, configuration boundaries, and evidence packets. Agreement should be examined at criterion level before aggregation, since matching property labels can conceal different E/N/U judgments. Disagreement analysis should examine actor boundaries, evidence applicability, criterion completeness, the distinction between a scoped negative witness and unresolved evidence, and the conditions for continuation and governed reuse. Sensitivity studies should then vary system boundaries and evidence availability.

Comparative diagnosis requires a documented assessment procedure based on the composition of established approaches in Section 2.4. Assessors should receive the same case information and objectives, matched familiarization and evidence access, and randomized assignment to assessment conditions. Independently developed cases should include supported transitions, violated relationships, and deliberately incomplete evidence.

Reference judgments should follow each case's operational contract, source records, and controlled observations where available. Diagnostic correctness should concern what that evidence justifies, irrespective of either procedure's terminology or agreement with the six-property labels. Outputs should identify the affected decision, supporting evidence, unresolved or violated relationship, and proposed follow-up check. Evaluation should separate correct findings, omissions, unsupported absence claims, appropriate evidence requests, and the relevance of follow-up checks. Assessment time and evidence-gathering effort should be recorded separately. Table 6 illustrates this output format; an independent comparison requires new cases and assessor observations.

Operational effects and architectural costs.

Controlled implementations should vary whether a relevant relationship is present, weakened, or removed, then introduce conditions under which it should matter. Examples include contested resources, executor replacement, uncertain effects, policy revision, and reuse under changed conditions. These experiments should relate observed behavior to the assessed relationship and record interactions with other mechanisms.

The checks in Table A9 can supply candidate scenarios. Executable tests require concrete record schemas, interface behavior, event schedules, and independently observable outcomes. Fault injection should include delayed confirmations, changed resource commitments, version mismatches, and overlapping execution capability. Observed admission, pending, rejection, reconciliation, and obligation statuses can then be compared with the analytical expectations. Execution results and implementation conformance remain to be established.

Existing benchmarks can contribute environments and evaluators. AgentBench covers interactive reasoning and decision making [81]; $\tau$-bench combines user interaction, policy constraints, tool use, state verification, and repeated-trial reliability [26]. OSWorld evaluates tasks through external applications [3], while WeaveBench examines hybrid GUI–CLI/code workflows and trajectory evidence, including fabricated visual evidence [4]. Architectural studies need episodes that also expose intermediate obligations, responsibility changes, stale or contradictory state, uncertain outcomes, component replacement, and later reuse.

Task success should be accompanied by process measures selected for the domain: commitment satisfaction, resource conflicts, reconciliation errors, incorrect retries, authority violations, handoff cor-

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



rectness, recovery time, adaptation regressions, and supervisory effort. Comparative implementations should also measure storage and synchronization overhead, reconciliation delay, review workload, and change-activation complexity. These costs determine whether improvements in process integrity are practical.

Longitudinal realization and reproducibility.

Controlled stress scenarios should progress toward naturalistic deployments [82], including repeated work episodes under changing participants, policies, services, and operating conditions. This permits assessment of later knowledge use, responsibility preservation, and the benefits or regressions of accepted changes.

Reference implementations can expose the operational objects and contracts through workflow engines, agent harnesses, distributed services, domain infrastructure, or Agentic Operating System services. Comparisons should examine whether the required relationships survive these implementation choices. Studies should report assessed boundaries, component versions, enabled mechanisms, environment assumptions, workloads, codebook version, and supporting evidence. Retained traces and evaluation artifacts should make configuration changes distinguishable from changes in evidence or assessment, especially when external models, tools, and policies evolve.

## 8. Conclusions

Agentic AI 2.0 organizes established architectural concepts around the maintenance of ongoing goal-directed work. Its six properties, reference architecture, acceptance contracts, and criterion-level codebook connect operational responsibilities to the evidence needed for commitments, actions, continuation, and reuse.

The constructed analysis identifies decision-specific consequences of these relationships: local feasibility can leave joint commitments unresolved; effect reconciliation and execution-right control address separate continuation conditions; and retained skill versions require governed activation and dependency binding for later use. Static configuration profiles distinguish established criteria from those contradicted by scoped negative witnesses or left unresolved by the inspected evidence. Candidate evaluation and cross-work reuse in the worked example remain conditional.

These results provide an inspectable basis for architectural analysis and design. The next step is independent evaluation of assessment reliability and diagnostic utility, followed by controlled and longitudinal studies of concrete implementations. Such studies should relate architectural choices to operational outcomes and their computational, communication, and supervisory costs.

## Appendix A. Codebook v1.0 and Assessment Details

Tables A1 and A2 specify the 21 constituent criteria. Their identifiers link the required relationships to evidence establishing E and counterevidence establishing N. Unresolved cases receive U under Section 3.3; property labels follow Equation 1.

### *Appendix A.1. Assessment Records and Reassessment*

Each record identifies the assessed configuration, workload and work-episode boundaries, versions and enabled components, actor assignments, supported changes, and relevant authority, trust, and environmental conditions. It records the criterion identifier, codebook version, claim basis, applicability conditions, source locator and version, supporting finding, counterevidence, unresolved questions, assigned code, and rationale. Locators identify the relevant passage, code path, trace, or evaluation artifact. Implementation reports retain their provenance separately from direct source inspection or execution by the assessor.

For source-based profiles, criterion vectors follow ascending criterion identifiers within each property. The record separates what the source establishes from its interpretation under the criterion and

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



identifies the configuration-specific path or constraint supporting any negative witness. Appendix D provides the records for the configurations analyzed in Section 6.4.

Reassessment distinguishes changes in the system from changes in scope, evidence, or classification. Changes to a criterion set, its interpretation, or the aggregation rule require a new codebook version and explicit reassessment. Preserving criterion-level findings allows alternative groupings to be examined without attributing a changed classification to a changed implementation.

*Appendix A.2. Constituent Criteria*

**Table A1.** Codebook v1.0: Maintained Process State, Collective Coordination, and Shared Operational Knowledge

| ID | Required operational relationship | Evidence establishing E | Counterevidence establishing N |
|---|---|---|---|
| **P1: Maintained Process State** | | | |
| P1.1 | Work-associated state is retained and made available to subsequent execution episodes. | A state-retention and retrieval path tied to the work, at the assessed claim basis. | Included execution episodes discard required prior work state or cannot retrieve it. |
| P1.2 | The work account identifies accepted goals and completion conditions, progress, dependencies, unfinished obligations, and decision-relevant uncertainty. | Representations and uses of these relations for the declared undertaking and workload. | A required relation is excluded, or an unresolved condition is represented as settled. |
| P1.3 | Accepted state updates follow identifiable acceptance and version rules that distinguish received information from the current operational account. | Update rules and attribution linking evidence, acceptance decisions, and applicable versions. | Unaccepted or superseded information becomes operative without the required acceptance treatment. |
| **P2: Collective Coordination** | | | |
| P2.1 | Distinct agent actors make attributable, interdependent contributions to the same work. | Actor–role assignments and consequential contributions under the boundary in Section 3.1. | The complete assessed arrangement has only one agent actor, or only internal component specialization. |
| P2.2 | Responsibilities, dependencies, and accepted commitments are allocated and attributable to the participating actors. | Allocation and acceptance relations identifying the responsible actors and dependent work. | A required obligation lacks an assignment, or a proposal becomes a commitment without acceptance. |
| P2.3 | Included conflicts or changed conditions lead to authorized revision, deferral, or release of affected plans and commitments. | A resolution path connecting the triggering condition to an accepted disposition of affected work. | Incompatible commitments remain concurrently operative, or required revisions bypass their acceptance conditions. |
| **P3: Shared Operational Knowledge** | | | |
| P3.1 | Information is made usable across distinct operational actors in carrying out their assigned responsibilities. | Identified contributing and consuming actors, information access, and its operational use. | The complete arrangement confines retention and retrieval to one actor's internal components. |
| P3.2 | Consumers can distinguish relevant information status, provenance, temporal context, and applicable versions. | Information records and consumption rules preserving these distinctions. | Information loses a required distinction or version context while being relied on operationally. |
| P3.3 | Use follows applicable access and validity conditions, including treatment of conflicting or superseded information. | Access, validity, and conflict-handling relations connected to the consuming responsibility. | Disallowed, conflicting, or superseded information is used contrary to the applicable conditions. |

E requires the complete criterion to be established at the declared claim basis and scope. The evidence descriptions admit equivalent realizations. N requires a configuration-matched negative witness; absence of a named mechanism or documentation silence is insufficient. Otherwise the criterion receives U under Section 3.3.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



**Table A2.** Codebook v1.0: Governed and Grounded Action, Computational Continuity, and Feedback-Driven Adaptation

| ID | Required operational relationship | Evidence establishing E | Counterevidence establishing N |
|---|---|---|---|
| **P4: Governed and Grounded Action** | | | |
| P4.1 | An action is attributable to a responsible actor and applicable authority with an identifiable source and scope. | Authority assignments and decision rules connected to the assessed action. | The action proceeds beyond its authority or relies on authority established only by its own proposal. |
| P4.2 | Specified execution preconditions and constraints govern dispatch, rejection, or deferral. | A decision path linking applicable conditions to whether the action proceeds. | An included action bypasses a required precondition or proceeds despite a failed constraint. |
| P4.3 | The action path reaches the declared environment and returns observations attributable to the requested operation. | An execution interface and environment-linked observations for the assessed operation. | A placeholder or disconnected path substitutes for the represented environmental operation. |
| P4.4 | Outcome acceptance uses action-relevant evidence; failures and unresolved effects remain explicit in subsequent decisions. | Confirmation criteria and dispositions for success, failure, and uncertainty. | Success is accepted from dispatch alone, or dependent work treats an unresolved effect as confirmed. |
| **P5: Computational Continuity** | | | |
| P5.1 | At least one identified interruption or execution-component change has a resumption path for work-associated state. | A named change or interruption and its state-resumption path, within the declared scope. | The claimed case loses required state, or the configuration affirmatively excludes every such path. |
| P5.2 | Continuation identifies the undertaking and its accepted terms, and accounts for unfinished obligations and uncertain prior effects. | Process references, amendments, obligation dispositions, and action reconciliation or applicable effect guarantees. | A required obligation is dropped, or an uncertain effect is repeated without adequate reconciliation. |
| P5.3 | The receiving configuration is compatible with the state, interfaces, resources, and conditions needed for the remaining work. | Compatibility checks or established constraints covering the declared transition. | Continuation uses an incompatible state, interface, resource, or configuration dependency. |
| P5.4 | Responsibility and authority to continue are retained or transferred explicitly, with overlapping execution rights resolved and the disposition recorded. | Assignments, authority conditions, and applicable control or guarantees for prior executors. | Work resumes without applicable authority or leaves conflicting execution rights unresolved. |
| **P6: Feedback-Driven Adaptation** | | | |
| P6.1 | Feedback informs identifiable candidate changes or retained experience intended for later work. | A path from feedback to a retained candidate or reusable experience object. | The complete arrangement excludes cross-work candidate production or experience retention. |
| P6.2 | Candidate admission depends on evaluation against declared acceptance conditions, with rejection or revision possible. | Evaluation inputs, an identifiable candidate, acceptance conditions, and the resulting disposition. | Candidates are admitted independently of a required evaluation or despite its rejection. |
| P6.3 | Activation identifies the accepted version and applicability, and governs its adoption for affected work and dependencies. | Version-linked acceptance and activation decisions, including the affected-work scope and dependencies. | An active replacement lacks the required connection to accepted versions, applicability, or affected dependencies. |
| P6.4 | Reuse is connected to the retained version and application context, with outcome feedback informing subsequent retention or change decisions. | Retrieval, use, and outcome records linked to the reused version and follow-up decisions. | The complete reuse path excludes the required outcome linkage or subsequent feedback treatment. |

The criteria specify operational relationships rather than mandatory technologies. Authority may be delegated; compatibility may follow from justified constraints; and effect handling may use appropriate reconciliation or execution guarantees. A dedicated approval service, world model, rollback mechanism, or named evaluation suite is not universally required. Evidence and counterevidence are interpreted at the declared claim basis under Section 3.3.

### *Appendix A.3. Grouping Choices*

The six-property grouping separates distinguishable assessment responsibilities. P1 and P3 could be combined as information management; their separation distinguishes the accepted condition of an undertaking from information used across responsibilities. P4 could be split into authorization

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



and outcome grounding; their grouping attaches both judgments to the same action. P5 could be treated as a cross-cutting condition; it is retained explicitly to examine how state, commitments, effects, compatibility, responsibility, and authority connect across a component change. P6 could be split into experience retention, evaluation, and change governance; their grouping follows the path to controlled reuse.

Alternative splits and merges change the criteria aggregated under a property while preserving their underlying evidence. For example, established authorization and unresolved outcome confirmation can yield PS for grouped P4 when its other criteria contain no N; a confirmed violation instead yields NS. Splitting authorization from grounding produces separate labels for these findings. Comparison of groupings therefore requires retention of criterion-level judgments and explicit aggregation rules.

### *Appendix A.4. Boundary Cases and Realization Choices*

Actor boundaries.

A single agent actor with several prompts, planners, critics, or stores may maintain state and recover from interruption while retaining responsibility within one assignment. P2 requires interdependent contributions from distinct agent actors, and P3 requires information use across distinct operational actors. Several actors can share a model or runtime; the assessment follows their assignments and consequential relationships.

Fixed procedures and selective adaptation.

A configuration may revise its current plan while deliberately keeping reusable procedures fixed. Affirmative evidence that the complete arrangement excludes cross-work adaptation distinguishes this case from incomplete documentation. Where an adaptation pathway exists, evaluation may justify retaining the current version or rejecting a candidate. The defining requirements concern the pathway and its conditions, rather than mandatory adoption in every episode.

Human participation.

An approval step supports the authorization relation established by the human's decision scope. Its contribution to other properties depends on the responsibility, information, execution, and outcome relationships it connects. Direct human decisions and delegated authority are assessed under the same requirement to identify their source and scope.

Supporting mechanisms.

Checkpointing supplies retained execution state; its contribution to P5 depends on the continuation conditions in Section 3.2.5. Retained traces supply material for adaptation; P6 concerns their connection to evaluated change and later use. APIs, graphical interfaces, visual reasoning, multimodal interaction, real-time processing, physical controllers, and predictive world models are workload-dependent realization choices. Agentic OS services or existing infrastructure can supply scheduling, memory, access control, and lifecycle support. Their contribution is assessed through the operational relationships specified by the criteria.

## Appendix B. Supporting Mechanisms, Deployment, and Change Details

This appendix supplies implementation choices and crosswalks for the reference architecture. The mechanisms illustrate ways to realize its responsibilities; assessment uses the unchanged Codebook v1.0 in Appendix A.

### *Appendix B.1. Capability Mechanisms and Property Links*

Table A3 identifies selected property relationships for the seven capability families in Figure 2.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



**Table A3.** Selected capability-to-property relationships.

| Capability family | Related properties |
|---|---|
| Continuous Perception and Multimodal Interaction | P1, P4 |
| Interactive and Visual Reasoning | P2, P4 |
| Social Reasoning and Collective Planning | P1, P2 |
| Shared Memory and Knowledge Accumulation | P1, P3, P5, P6 |
| World Knowledge and World Models | P2, P3, P4 |
| Grounded and Governed Collective Action | P4, P5 |
| Lifelong Learning and Skill Evolution | P6 |

Links indicate selected contributions, with sufficiency determined by the constituent criteria. Governance and Computational Continuity constrain operational use across capabilities.

### Appendix B.1.1. Continuous Perception and Multimodal Interaction

Acquisition can be event-driven, periodic, or on demand. Inputs include documents, conversations, interfaces, sensors, and execution traces. Multimodal interpretation may require representation, alignment, or fusion [83]. Observations retain source, event time, reception or processing time, and relevant uncertainty. Event-time distinctions help assess late or out-of-order information [84]; provenance connects derived records to their sources and producers [85]. Freshness, source authority, and scope inform conflict handling. Acquisition and interpretation produce candidate updates whose acceptance follows Table 4. VisualWebArena and OSWorld illustrate the interaction between multimodal observations and interface actions [3,86].

### Appendix B.1.2. Interactive and Visual Reasoning

Interactive reasoning develops and revises alternatives using tools, participants, and external representations. ReAct interleaves reasoning and action [1]; LATS searches alternative reasoning–action trajectories using feedback and value estimates [23]. Visual reasoning uses spatial or visual representations during problem solving, distinct from interpreting visual input or displaying an already derived result. Distributed cognition provides a basis for this use of external artifacts [12]; Visual Sketchpad supplies drawing tools used in subsequent reasoning [87]. Operational outputs comprise alternatives, evidence, assumptions, qualified predictions, and unresolved questions. These support downstream assessment without requiring disclosure of a model's complete internal chain of thought.

### Appendix B.1.3. Social Reasoning and Collective Planning

Proposal, critique, negotiation, feasibility analysis, and plan repair connect contributions from roles with different information, resources, and authority. SharedPlans and STEAM address collaborative plans, team commitments, and responsibility revision [7,8]; AutoGen illustrates conversation-based composition of agents, tools, and human participants [24]. Plans retain versions, assignments, dependencies, allocations, and unresolved conditions. Permissions and obligations constrain their acceptance [9]. Collective Coordination is assessed through consequential, interdependent contributions by distinct agent actors under Section 3.1.

### Appendix B.1.4. Shared Memory and Knowledge Accumulation

Memory can retain operational state, episodic records, semantic knowledge, and procedures [2]. Generative Agents illustrate memory retrieval and reflection [72]; Agora represents shared artifacts with lineage and verification status [88]. Operational use preserves distinctions among observations, hypotheses, accepted claims, decisions, and outcomes, including links between source records and summaries. Read, write, publication, and acceptance authority can differ. Linked or federated stores may therefore expose role-specific views while retaining validity, provenance, and material conflicts.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



Handoff additionally requires the obligations, uncertain effects, and continuation checks specified in Section 5.3.

#### Appendix B.1.5. World Knowledge and World Models

World Knowledge represents relevant entities, relations, rules, and constraints. Ontologies and retrieval-augmented generation provide supporting mechanisms [89,90]. World Models estimate consequences through transition, process, constraint, simulation, or learned dynamics models. Dreamer illustrates learning and decision-making with imagined futures [91]; its actor and critic are algorithmic components whose separation has a different meaning from operational actor assignments. Where predictions are used, records identify model version, inputs, assumptions, applicability, and uncertainty. These predictions inform planning and can be compared with later outcomes while retaining their evidential status.

#### Appendix B.1.6. Grounded and Governed Collective Action

Execution channels include APIs, graphical interfaces, and specialized controllers. Toolformer studies learned API use [92]; WeaveBench examines interleaved GUI and CLI/code execution [4]. Runtime shielding offers a mechanism for restricting actions against specified safety constraints [93]. The selected channel determines which preconditions it can enforce and what its responses establish. Dispatch and outcome records follow Section 5.2; uncertain effects remain inputs to retry, recovery, and handoff decisions.

#### Appendix B.1.7. Lifelong Learning and Skill Evolution

Current-work revision, experience accumulation, and cross-work reuse have different scopes. Reflexion uses retained verbal feedback across attempts [29]; Voyager accumulates executable skills [6]. Agent Workflow Memory induces reusable workflows, while ExpeL accumulates insights and experiences [30,94]. A-Mem revises linked memory structures [31]. Search-R1 trains search-tool use through reinforcement learning [32], and the agentic reinforcement-learning survey synthesizes a broader range of training approaches [33]. These mechanisms supply candidates for reusable change. Their acceptance, activation, and later use follow Section 5.4, with reported results bounded by each source's configuration and evaluation.

### *Appendix B.2. Deployment Choices and Decision-Specific Consistency*

Coordination may be centralized, distributed, or replicated; information may reside in shared stores, federated repositories, or authoritative external systems. Model services can be shared across roles or specialized by task, with corresponding contention, routing, and compatibility requirements. APIs, GUI actions, and specialized controllers provide different execution and evidence interfaces. Cloud, edge, and hybrid placement determine resource and communication dependencies. Human participation requires specified decision scope, presented evidence, and recorded effects on the work.

Shared infrastructure can group scheduling, context, memory, storage, tool access, permissions, observability, and recovery services. AIOS illustrates kernel support for agent applications [43]; the AOS proposal groups lifecycle management, workflow coordination, routing, and assurance in a runtime and coordination plane [48]. A deployment assigns acceptance and reconciliation responsibilities alongside these services.

Consistency requirements attach to supported decisions. Invariant confluence analysis provides a precedent for relating coordination requirements to operations and application invariants [95]. Table A4 applies this decision-specific reasoning to the proposed architecture.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



**Table A4.** Consistency and availability conditions by operational use.

| Decision and relevant records | Conditions to establish |
|---|---|
| Retrospective analysis: Action Records; Observations and Evidence | Delayed synchronization may be acceptable with identifiable provenance, integrity, and missing coverage. Establish completeness where the analysis requires it. |
| Allocation: Plans and Commitments; resource state | Coordinate affected updates or use an allocation scheme preserving shared limits and existing obligations when commitments take effect. |
| Authorization: Roles, Policies, and Authority | Use authority valid for the decision's scope and time. Cached authority follows explicit expiry and revocation semantics, including during disconnected operation. |
| Retry, completion, or resumption: Action Records, evidence, state, commitments, and authority | Reconcile request-specific effects and establish evidence for required obligations. Resumption additionally establishes compatibility, responsibility, authority, and effective control of overlapping execution rights. |
| Reuse or activation: Skills and Change Records; configuration metadata | Establish applicable evaluation, authorization, compatible dependencies, and identifiable activation scope. Mixed versions require supported coexistence. |

Requirements apply jointly to the objects involved in a decision. A single object can support uses with different consistency and availability requirements.

Disconnected activity remains within previously established authority, resources, and valid dependencies. Reconnection requires reconciliation before such activity updates shared state or commitments. Decisions lacking a required basis follow the dispositions in Section 4.4.

Context compaction must preserve access to relevant commitments, authority, uncertainty, and versions. Diagnostic sampling can follow the declared scope, while evidence required for acceptance or outcome confirmation needs an appropriate retention guarantee. Supported replacement also preserves the interpretability of records, permissions, configuration references, and outstanding responsibilities.

*Appendix B.3. Crosswalk of Organizational Views*

The architecture relates six properties, seven capability families, five responsibility groups, nine operational-object classes, six transition families, and three loops. These organize different questions about the same configuration. Table A5 connects the properties to selected responsibilities and transitions; capability links appear in Table A3.

**Table A5.** Selected property-to-architecture crosswalk.

| Property | Responsibilities and principal records | Transitions and loop connections |
|---|---|---|
| P1 | State, Knowledge, and Evidence Management; Plan and Commitment Management. Goals and Constraints, Operational State, Observations and Evidence, Plans and Commitments. | Observation acceptance and state reconciliation support execution, coordination, and handoff. |
| P2 | Agent Roles and Contribution Scopes; Plan and Commitment Management. Roles, Policies, and Authority, Goals and Constraints, Plans and Commitments, relevant state. | Commitment acceptance and responsibility transfer connect coordination decisions to execution. |
| P3 | Agent Roles and Contribution Scopes; State, Knowledge, and Evidence Management. World Knowledge, evidence, relevant state and change records, and predictions where used. | Information use across actors supports all three loops, preserving status and acceptance conditions through updates, candidate formation, and adoption. |

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



**Table A5.** *Cont.*

| Property | Responsibilities and principal records | Transitions and loop connections |
|---|---|---|
| P4 | Agent Roles and Contribution Scopes; Execution and Outcome Reconciliation. Authority, plans, Action Records, evidence, and relevant state. | Admissible dispatch and outcome assessment connect execution to coordination outputs and return evidence or unresolved effects. |
| P5 | Agent Roles and Contribution Scopes; Plan and Commitment Management; State, Knowledge, and Evidence Management; Execution and Outcome Reconciliation. Goals, state, commitments, Action Records, authority, and configuration metadata. | Accepted responsibility and admissible resumption connect coordination and execution. Configuration changes can additionally require change acceptance and activation. |
| P6 | Change Evaluation and Acceptance; State, Knowledge, and Evidence Management; affected Agent Roles and Contribution Scopes. Skills and Change Records, evidence, Action Records, relevant knowledge and authority. | Candidate formation, acceptance, activation, and version-linked reuse connect experience to conditional changes in coordination and execution. |

Mappings are selective and many-to-many. Object names are shortened where their meaning follows Table 3. Configuration metadata belongs to the relevant records. Evidence for a property follows its constituent criteria and the composition conditions in Section 4.4.1.

### *Appendix B.4. Assurance Scope and Change-Specific Checks*

Assurance claims retain the scope of their evidence. Reported limitations of intrinsic self-correction concern reasoning without external feedback in the examined settings [96]; their application to external outcomes requires separate environmental evidence. Model checking establishes specified temporal properties of a model [97]. An implementation claim additionally identifies that model's assumptions and its relationship to the deployed system.

Recent agent-system work addresses complementary recovery conditions: checkpoint and resume conformance [34], security-relevant dependencies under rollback [35], transactional tool execution [36], workflow retries and orphaned jobs [37], and event-sourced handoff [38]. These contributions inform implementation choices. Their coverage of commitments, uncertain effects, and continuation authority is assessed for the declared configuration using Section 5.3.

**Table A6.** Change-specific checks for affected ongoing work.

| Changed object | Acceptance basis and affected-work disposition |
|---|---|
| Goal or completion condition | Establish change authority and revised terms under the declared identity rule. Record the effective version, affected commitments and effects, and any successor reference or inherited obligations. |
| Plan or resource allocation | Establish proposal version, joint feasibility, dependencies, and authority at commitment acceptance. Identify retained, revised, or released commitments. |
| Knowledge or world model | Establish provenance, assumptions, validity, evaluation scope, and active version. Reassess dependent plans, predictions, and decisions. |
| Skill or workflow | Establish version-bound evaluation and acceptance, applicability, compatible dependencies, and activation authority. Record actual consumers, supported earlier versions, and version-linked outcomes. |
| Agent, model, or execution component | Establish process reference, accepted terms, compatible state and interfaces, uncertain-action treatment, and continuation authority. Record retained or transferred assignments and resolve overlapping execution rights. |
| Policy or authority | Accept under currently valid authority with effective time, scope, and revocation conditions. Review affected permissions, obligations, commitments, and prior decisions; re-authorize where required. |

These are requirements of the proposed architecture. A revision to current work constitutes feedback-driven adaptation only when the relevant experience, acceptance, and later-use relationships are established under P6.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



Recovery assumptions depend on the action model. Zhai et al. [98] analyze specification revision where incompatible irreversible effects can prevent satisfaction of revised requirements. Mohammadi and Bindschaedler [99] propose fleet-level accounting and admission control for irreversible exposure. Chan et al. [47] discuss rollback infrastructure and practical limits on undoing effects. Their respective scopes motivate explicit treatment of reversibility and compensability at admission, alongside the outcome and reconciliation requirements in the main text.

## Appendix C. Trace Specification and Controlled Checks

This appendix records the constructed trace underlying Section 6. The ledgers preserve decision dependencies and witness requirements; their successful $O_1$ states are stipulated. T11–T12 retain conditional evaluation and reuse.

### *Appendix C.1. Assumptions and Trace Records*

The scenario adopts six assumptions:

1. roles and authority belong to one organizational context;
2. relevant environmental evidence is obtainable, without assuming action success;
3. information may be delayed, incomplete, or conflicting;
4. predictive representations cover only the scenario's required relations and retain their assumptions;
5. changes to materials, machines, and execution components are explicitly introduced analytical branches; and
6. process, resource, organizational, and interface details are restricted to the architectural questions examined.

Designated authority assignments and record sources form the reference basis, subject to observational uncertainty. Compromise of those sources and other model or infrastructure changes require separate cases and evidence. Each branch retains the initial conditions except where explicitly changed.

A transition record contains its identifier, process reference, before-status, proposing or executing actor, decision authority, input objects and versions, witnesses, guard disposition, after-status, and remaining obligations. Objects may use separate versions and storage transactions. The $W$-references specify claims, sources or responsible actors, work and object versions, temporal scope, and qualifications. They denote required evidence rather than collected execution artifacts.

The trace retains $O_1$ through $P_1 \to P_2$ and $E_1 \to E_2$, with unchanged accepted terms and the same Execution actor. $O_2$ has a distinct process identity. Each receiving transition applies Equation 2; an unresolved required condition keeps dependent work pending, while a demonstrated violation requires rejection or revision of its current basis. Existing effects and obligations retain their disposition requirements.

### *Appendix C.2. Checking Conditions*

**C1.** *Identity and dependency binding.* Decisions identify the intended process, accepted terms, and applicable object versions. Changed dependencies are reassessed before dependent use.

**C2.** *Claim and outcome separation.* Proposals, predictions, accepted state, requests, confirmed effects, and completed stages retain distinct statuses. A witness supports its established claim.

**C3.** *Authority at the receiving transition.* The actor and decision authority have the required scope for the commitment, action, change, or continuation.

**C4.** *Joint feasibility and conditions at use.* Commitments satisfy shared constraints; changing conditions are protected or checked at the operational point covered by the contract.

**C5.** *Continuation with reconciled effects.* Resumption requires an adequate account of prior effects, compatible configuration, accepted responsibility, and resolved overlapping execution rights. Investigation may precede effect-producing resumption.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



**C6.** *Obligation accounting.* Replanning, handoff, and closure preserve unfinished work, unresolved effects, and commitments until explicitly disposed of.

**C7.** *Version-bound adaptation and reuse.* Formation, evaluation, acceptance, activation, and use identify the relevant version. $O_2$ establishes its own applicability and authority basis and retains outcome links.

Identity and obligation records concern P1.1–P1.3; actor contributions and commitments concern P2.1–P2.3; qualified cross-actor information use concerns P3.1–P3.3. Action admission and outcome assessment concern P4.1–P4.4, and T07–T08c expose P5.1–P5.4. T10–T12 specify P6.1–P6.4 relationships, with evaluation and reuse evidence still unresolved. This mapping locates criterion evidence requirements within the construction.

*Appendix C.3. Transition Ledgers*

**Table A7.** Constructed ledger: work establishment through replanning.

| Step | Input, witness, and decision | Result and remaining obligations |
|---|---|---|
| T00 | $O_1$ is assigned. $W_{\text{terms}}$ identifies accepted stages, deadline, budget, quality conditions, and authority. Production Planning records them under designated authority (C1, C3, C6). | Work is established; preparation, processing, quality control, and closure remain outstanding. Resource feasibility awaits assessment. |
| T01 | $W_{\text{state}}$ supplies sourced, temporally qualified material, machine, and progress observations. Inventory / Procurement, Maintenance, and Execution validate claims within their scopes, preserving uncertainty (C1, C2). | Accepted Operational State supports specified planning uses; unresolved availability or progress remains explicit. |
| T02 | Planning combines accepted state, order terms, and actor contributions into a versioned proposal, checking common work references and affected responsibilities (C1, C6). | A candidate plan records dependencies; proposed allocations remain uncommitted. |
| T03 | $W_{\text{plan}}$ identifies the candidate's assumptions. Planning and resource-responsible actors inspect dependencies, predictions, feasibility, and unresolved conditions (C2, C4). | Established conditions support an acceptance decision. Unresolved dependencies remain conditional; predictions retain their status. |
| T04 | For proposed use of $M_1$, $W_{P_1}$ binds one proposal version to availability, authority, and effective allocation. Commitment authorities, including the Human Supervisor where required, check joint feasibility at commitment (C1, C3, C4). | $P_1$ is committed in the selected branch, with allocations and obligations recorded. Execution retains its own admission requirements. |
| T05 | $W_{\text{progress}}$ supplies the dispatch basis and environmental evidence. Execution checks admission; the verification decision accepts supported stage claims (C2–C4). | Preparation is complete and processing on $M_1$ has started. Processing completion and quality control remain outstanding. |
| T06 | Following $M_1$'s failure, $W_{\text{revision}}$ identifies valid prior outputs, remaining work, $M_2$ conditions, and affected commitments. Execution reconciles progress; Planning and authorities assess substitution (C1–C4, C6). | $P_2$ replaces the affected part of $P_1$. Commitments are retained, revised, or released; remaining processing and quality control are preserved. |

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



**Table A8.** Constructed ledger: continuation, closure, and conditional reuse.

| Step | Input, witness, and decision | Result and remaining obligations |
|---|---|---|
| T07 | $E_1$ issues $a_2$ under $P_2$ for remaining processing on $M_2$. Execution retains its admission basis, work reference, plan, and available interface status (C1, C2, C6). | The dispatch effect is unresolved. Retry eligibility and processing completion remain unestablished. |
| T08a | $W_{\text{dispatch}}$ correlates an enterprise record with $a_2$, $O_1$, its stage, and $M_2$. The responsible verifier assesses the particular dispatch effect (C1, C2, C5). | The selected branch confirms dispatch and leaves $a_2$ unrepeated. Processing completion requires separate evidence. |
| T08b | $W_{\text{handoff}}$ covers $E_2$'s compatibility, retained actor responsibility, continuation scope, and overlapping execution control. Continuation authority establishes permitted activities and the treatment of $E_1$, outstanding requests, and obligations (C3, C5, C6). | The specified handoff conditions are established for $E_2$. Any remaining gap restricts activity to independently admissible operations. |
| T08c | Execution checks the compatible T08a–T08b results for the same work, current $P_2$ dependencies, and conditions at use (C1, C3–C6). | Dependent work resumes without repeating $a_2$. Changed dependencies or unresolved handoff conditions keep that work pending. |
| T09 | $W_{\text{closure}}$ covers remaining stage outcomes, order constraints, and commitments. Responsible actors account for obligations; closure authority assesses the complete order conditions (C2, C3, C6). | $O_1$ completes in the selected branch. Unmet conditions would prevent successful closure and retain responsible owners. |
| T10 | Planning coordinates $s_1$ formation from the retained $O_1$ trace with contributing actors. Check the identified version, trace links, assumptions, and applicability (C1, C7). | $s_1$ is a candidate for evaluation; acceptance and activation remain separate decisions. |
| T11 | $W_{\text{evaluation}}$ must supply results for the same candidate version, declared criteria, and applicability. The evaluator and change authority record the disposition (C1, C3, C7). | Evaluation results are absent from this construction. Acceptance remains conditional on a completed, supporting evaluation and authorized decision. |
| T12 | Use in $O_2$ requires the accepted version, $W_{\text{use}}$, and activation. Responsible actors check resources, interfaces, evidence, and authority; actual use links the version to decisions, actions, and feedback (C1, C3, C4, C7). | Reuse remains conditional. Retrieval supplies a candidate for consideration; actual use and improvement require their own evidence. |

T08a and T08b may proceed separately; T08c requires their compatible combination. T11–T12 specify requirements without supplying evaluation or reuse results.

### *Appendix C.4. Changed-Condition Checks*

Each check changes one decision-relevant condition while retaining the other branch assumptions. The reader can trace the changed input to its step, proposed status, and required disposition. Admitting the transition despite the stated gap violates the indicated condition. A pending disposition preserves the requirement while leaving eventual completion unresolved.

In the $M_3$ branch, the Human Supervisor can approve the change only within its assigned scope; otherwise escalation continues to an appropriate authority. Approval identifies permitted work and conditions, with consistent authority, plan, and commitment references. Resource conditions changed during approval require reassessment before commitment and use. Denial leaves the proposal inadmissible; a pending decision leaves dependent work pending.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



**Table A9.** Analytical checks under changed conditions.

| Check | Changed condition | Required disposition | Conditions |
|---|---|---|---|
| K1 | At T06, the $M_2$ witness concerns another proposal version or an allocation basis changed by another commitment. | Withhold $P_2$ commitment on that basis; reassess allocation and preserve existing obligation statuses. | C1, C4 |
| K2 | At T06, reconciliation leaves an $M_1$ output's validity unresolved. | Retain that uncertainty and require revalidation before using the output to satisfy a dependent stage. | C2, C6 |
| K3 | At T08a, the record concerns another $O_1$ request and cannot be correlated with $a_2$. | Keep $a_2$ unresolved; confirmation and retry eligibility remain unestablished. | C1, C2, C5 |
| K4 | T08a confirms dispatch, but T08b leaves $E_1$'s overlapping effect-producing capability uncontrolled. | Withhold dependent resumption at T08c. Permit investigation only within its authorized scope. | C3, C5 |
| K5 | At T09, processing is complete but required quality-control evidence is absent. | Withhold successful closure; retain the quality-control obligation and responsible actor. | C2, C6 |
| K6 | T11 evaluates $s_1$, while acceptance is requested for changed version $s_2$. | Assess $s_2$ and affected dependencies before acceptance; the earlier evaluation covers its identified version. | C1, C7 |
| K7 | At T12, the skill version is accepted but $M_2$ availability for $O_2$ is unknown. | Keep use pending until the receiving resource and commitment conditions are established. | C4, C7 |
| K8 | The $M_3$ branch establishes technical feasibility without an applicable authority change. | Withhold commitment and dispatch; retain the candidate for an authorized decision or choose an admissible alternative. | C3 |

*Appendix C.5. Supporting Counterexamples*

The allocation counterexample remains in Section 6.3. The following cases preserve the other conditional arguments. A later control may contain a gap before an external effect occurs.

*Agreement beyond authority.* Inspection is mandatory, and the participating agents lack authority to waive it. They nevertheless accept a plan omitting inspection. If Execution follows that revision without checking the unchanged obligation, completion violates the accepted terms. The missing check binds commitment acceptance to authority over that obligation.

*Prediction accepted as an outcome.* A model predicts successful $M_2$ processing before execution. If Operational State records completed processing and dependent work uses that record, the transition lacks evidence of the particular event. State acceptance must distinguish predictive support from environmental outcome evidence.

*Retry after an unreconciled effect.* An external dispatch has occurred, but the checkpoint predates confirmation. A receiving executor treats the missing record as non-execution and retries through an interface that accepts a new dispatch without duplicate suppression. Under those assumptions, the retry creates a second effect. Reconciliation or applicable effect guarantees must precede another dispatch.

*Overlapping execution after handoff.* Both outgoing and incoming instances can act on the same obligation after record transfer. Accurate state alone leaves conflicting actions possible. The continuation decision must resolve their execution scopes and effective rights. This addresses who may act next, separately from establishing prior effects.

*Incomplete continuation summary.* A summary omits a pending inspection or unresolved dispatch, and the receiver relies on it without consulting source records. The omission can drop an obligation or trigger an unjustified retry. Continuation requires decision-relevant completeness through retained information or traceable authoritative references, while allowing a bounded immediate context.

*An exceptional response reused as a general rule.* A successful $O_1$ response becomes an instruction to switch to $M_2$, stripped of availability, suitability, and authority conditions. In $O_2$, $M_2$ is already allocated. Accepting the recommendation without current resource checks creates a conflict; a down-

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



stream allocation check could contain it. The skill must retain its version and applicability conditions, with reassessment at use.

*Trace retention without an experience-use mechanism.* The complete assessed arrangement offers retained traces as its only adaptation mechanism, but supplies no retrieval, candidate evaluation, applicability assessment, or connection from later use to outcomes. The records support retrospective inspection; the claimed cross-work experience-use relationship remains unsupported. This gap concerns the adaptation claim, independently of whether either order completes.

## Appendix D. Evidence Dossier for the Configuration Profiles

This dossier records the implementation-source assessment (basis I) conducted on September 29, 2026. Pins and locators identify the inspected material; line numbers refer to those versions. Vectors follow criterion order in Tables A1 and A2, with aggregation under Equation 1. Each record separates established relationships, scoped negative witnesses, and unresolved requirements. Actor boundaries follow Section 3.1. E concerns the inspected implementation relationship; operational reliability requires execution evidence.

Tables A10, A11, A12, and A13 provide the criterion records underlying the profiles in Table 5.

### *Appendix D.1. MetaGPT*

The source is MetaGPT v0.8.2, commit `df9bc18` [62]. The selected `generate_repo` path sets `implement=True`, `code_review=True`, `run_tests=True`, and `inc=False`. Tests are explicitly enabled above the source default. The scope includes ordinary role memory and serialized-team recovery, without an added cross-project learning service.

- **MG1:** `metagpt/software_company.py`, `generate_repo`, lines 16–75: flags, role construction, QA activation, and recovery entry.
- **MG2:** `metagpt/team.py`, lines 56–78 and 99–138: serialization, reconstruction, project requirement, and team execution. `metagpt/roles/role.py`, lines 88–155 and 369–444: retained state and recovery-related observation/action selection.
- **MG3:** `metagpt/environment/base_env.py`, lines 156–207: roles, message routing, and execution. `metagpt/roles/project_manager.py`, lines 26–38; `metagpt/roles/engineer.py`, lines 88–198 and 298–349: task dependencies and artifact production, revision, and use.
- **MG4:** `metagpt/utils/file_repository.py`, lines 46–104; `metagpt/utils/git_repository.py`, lines 163–192: persistence, dependencies, change inspection, and archiving.
- **MG5:** `metagpt/roles/qa_engineer.py`, lines 48–169; `metagpt/actions/run_code.py`, lines 92–146: tests, subprocess execution, output retention, and result routing.

**Table A10.** MetaGPT criterion records, basis I.

| Record | Vector; label | Sources | Criterion rationale |
|---|---|---|---|
| MG-P1 | `EUU`; PS | MG1, MG2, MG4 | P1.1: retained team, role, and project state with reconstruction. P1.2–P1.3: U for complete accepted terms, unfinished obligations, uncertainty, and state-acceptance/version rules. |
| MG-P2 | `EUU`; PS | MG1, MG3 | P2.1: identifiable roles with consequential artifact dependencies. P2.2–P2.3: U for complete accepted commitments and authorized resolution of conflicting or changed commitments. |
| MG-P3 | `EUU`; PS | MG3, MG4 | P3.1: cross-role use of messages, designs, tasks, and code. P3.2–P3.3: dependency records and archiving provide partial information, leaving complete status, provenance, temporal/version applicability, and validity treatment U. |
| MG-P4 | `UUEU`; PS | MG3, MG5 | P4.3: local tests and outputs linked to code/test context. P4.1–P4.2: U for complete authority and admission relations. P4.4: U for complete outcome acceptance and uncertainty handling beyond test results and summaries. |

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



**Table A10.** *Cont.*

| Record | Vector; label | Sources | Criterion rationale |
|---|---|---|---|
| MG-P5 | EUUU; PS | MG1, MG2 | P5.1: reconstruction from serialized team state. P5.2–P5.4: U for obligations and uncertain effects, receiving compatibility, continuation responsibility and authority, and overlapping execution rights. |
| MG-P6 | UUUU; II | MG1–MG5 | Within-project correction and artifact retention are present. The inspected evidence leaves cross-work candidate formation, admission evaluation, governed activation, and version-linked reuse feedback unresolved, with insufficient basis for E or a closed-path negative judgment. |

*Appendix D.2. LangGraph Email-Tool Example*

The application is the complete Python example under *Interrupts in tools*, pinned at documentation commit `f1eefff` [63]. Runtime sources are LangGraph 1.2.11, `langgraph-checkpoint` 4.2.0, and `langgraph-checkpoint-sqlite` 3.1.1 [64,100,101]. These pins define a static assessment basis rather than an executed environment lockfile.

- **LG1:** `src/oss/langgraph/interrupts.mdx`, lines 593–693: complete application, including `AgentState`, `send_email`, graph, checkpoints, and caller resume.
- **LG2:** `libs/langgraph/langgraph/types.py` at 1.2.11, `interrupt`, lines 851–974: suspension and reuse of supplied resume values.
- **LG3:** `libs/checkpoint/langgraph/checkpoint/base/__init__.py` at checkpoint 4.2.0, lines 38–146: metadata, channel versions, parent references, and pending writes.
- **LG4:** `langgraph/checkpoint/sqlite/__init__.py` in the SQLite 3.1.1 wheel: `setup`, lines 129–167; `get_tuple`, lines 191–293; `put` and `put_writes`, lines 387–483.

The inspected artifact is `langgraph_checkpoint_sqlite-3.1.1-py3-none-any.whl`, with published SHA-256 identifier `8505c54c94a658080525d7e6780fdd4e0c078ff2566b30d399c02cc9f9af1c63`.

**Table A11.** LangGraph example criterion records, basis I.

| Record | Vector; label | Sources | Criterion rationale |
|---|---|---|---|
| LG-P1 | EUU; PS | LG1, LG3, LG4 | P1.1: thread-associated persisted state. P1.2–P1.3: U for the complete operational-state account and acceptance rules beyond checkpoints and message history. |
| LG-P2 | NUU; NS | LG1 | P2.1: N for one model actor with deterministic tool dispatch. P2.2–P2.3: U. |
| LG-P3 | UUU; II | LG1 | P3.1–P3.3: U because the caller's operational-actor assignment and required cross-actor knowledge relationship are unspecified. |
| LG-P4 | UENN; NS | LG1 | P4.1: U for caller authority. P4.2: E for approval gating. P4.3–P4.4: N for the represented email effect; the placeholder reports success without an email action or delivery evidence. |
| LG-P5 | EUUU; PS | LG1–LG4 | P5.1: checkpoint-backed interrupt/resume. P5.2–P5.4: U for complete obligation, compatibility, and continuation-authority relationships beyond execution lineage. |
| LG-P6 | NUUU; NS | LG1 | P6.1: N because the complete application dataflow excludes a cross-work candidate-generation path. P6.2–P6.4: U. |

LG-P4 evaluates the represented email-send task. A contract limited to printing a simulated operation would define a different assessment scope.

*Appendix D.3. Voyager*

The source is commit `55e45a8` [65]. Automatic curriculum and critic modes control one Minecraft bot. Curriculum, action, critic, and skill components are internal functions of that actor; manual modes, additional actors, and external governance services are outside the configuration.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



- **VG1:** `voyager/voyager.py`, lines 104–164, 165–285, and 295–368: components, action generation, environment calls, feedback, resumption, skill admission, and progress.
- **VG2:** `voyager/agents/skill.py`, lines 33–100 and 114–127: reload, program assembly, same-name replacement, historical files, and retrieval.
- **VG3:** `voyager/agents/curriculum.py`, lines 46–70 and 329–367: persisted progress and cached knowledge. `voyager/agents/critic.py`, lines 27–77 and 91–138: observation-conditioned task assessment and success disposition.
- **VG4:** `voyager/env/bridge.py`, lines 105–162; `voyager/utils/record_utils.py`, lines 27–75: environment requests, observations, resets, and task-linked events.

**Table A12.** Voyager criterion records, basis I.

| Record | Vector; label | Sources | Criterion rationale |
|---|---|---|---|
| VG-P1 | `EUU`; PS | VG1–VG4 | P1.1: retained progress, skills, knowledge, and event records. P1.2–P1.3: U for complete work terms, obligations, uncertainty, and accepted state versions. |
| VG-P2 | `NUU`; NS | VG1 | P2.1: N because internal components control one bot rather than distinct operational agent actors. P2.2–P2.3: U. |
| VG-P3 | `NUU`; NS | VG1–VG3 | P3.1: N because retrieval and information exchange remain within one actor. P3.2–P3.3: U. |
| VG-P4 | `UUEU`; PS | VG1, VG3, VG4 | P4.3: generated programs reach the environment bridge and return task-linked observations. P4.1–P4.2: U for complete authority and dispatch relations. P4.4: U for complete action-outcome and uncertainty treatment beyond critic feedback. |
| VG-P5 | `EUUU`; PS | VG1–VG4 | P5.1: retained-state reload and resumption. P5.2–P5.4: U for obligations, uncertain effects, receiving compatibility, responsibility, and execution rights. |
| VG-P6 | `EENU`; NS | VG1–VG4 | P6.1: feedback-driven revision and retained skills. P6.2: critic-gated admission. P6.3: N for active same-name replacement without acceptance of affected dependent use. P6.4: U for the complete link between reused version, context, and subsequent evaluation. |

The P6.3 witness follows three linked operations. `SkillManager.add_new_skill`, lines 71–99, removes an existing name from the retrieval index and replaces its active code entry. Historical files can receive suffixes such as `V2`, while `SkillManager.programs`, lines 52–59, assembles the active collection. `Voyager.step`, lines 213–218, supplies that collection to environment execution. An existing program calling the replaced skill by name can consequently resolve to changed code through this path, without acceptance covering its dependent use. This static witness establishes the scoped version-binding gap; observed regressions and replacement failure rates require separate execution evidence.

*Appendix D.4. AIOS Infrastructure Boundary Case*

The source is AIOS v0.2.2, commit `57046b5` [66]. Context management is disabled and FIFO scheduling selected. The assessment binds no particular agent application, workload, or organizational authority assignment.

- **AI1:** `aios/config/config.yaml`, lines 58–73: context-management and service settings.
- **AI2:** `runtime/launch.py`, lines 207–318: scheduler, components, and agent-factory integration; lines 449–519: submission and execution status; lines 567–597: service-query dispatch.
- **AI3:** `aios/scheduler/fifo_scheduler.py`, `FIFOScheduler._execute_syscall`, lines 99–151: syscall execution and status handling.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



**Table A13.** AIOS infrastructure criterion records, basis I.

| Record | Vector; label | Sources | Unresolved application relationship |
|---|---|---|---|
| AI-P1 | UUU; II | AI1–AI3 | Application work state, accepted terms, and state-acceptance rules beyond execution identifiers and job/syscall status. |
| AI-P2 | UUU; II | AI2, AI3 | Selected operational actors, interdependent commitments, and conflict-resolution rules beyond submission and scheduling. |
| AI-P3 | UUU; II | AI1, AI2 | Cross-actor information use, status conventions, and validity rules for the available memory and storage services. |
| AI-P4 | UUUU; II | AI2, AI3 | Application authority, admission conditions, intended effects, and outcome acceptance for tool dispatch and responses. |
| AI-P5 | UUUU; II | AI1–AI3 | Application continuation paths, obligation and effect reconciliation, receiving compatibility, and authority. |
| AI-P6 | UUUU; II | AI1–AI3 | Application feedback, candidate formation, evaluation, activation, and reuse relationships. |

All criteria remain U for the unspecified application. Infrastructure availability alone leaves their application-level realization unresolved.

## Appendix E. Domain-Specific Implementation and Evaluation Details

**Table A14.** Domain-specific information and evaluation priorities for the prospective application profiles. These requirements guide implementation and testing; support judgments require evidence from a bounded operational configuration.

| Requirement | Smart Productivity Workforce | Adaptive Manufacturing | Distributed Mobility |
|---|---|---|---|
| **Maintained work state** | Progress, accepted decisions, pending approvals and obligations, unresolved questions, artifact versions, and responsibility. | Machine and material state, work in progress, resource assignments, production commitments, inspection results, and quality status. | Location, capacity, route progress, requests, assignments, service commitments, traffic, communication status, and observation timestamps. |
| **Forecasts used in planning** | Deadline risk, workload, assignment feasibility, and process bottlenecks. | Processing duration, resource availability, equipment condition, quality risk, and reallocation effects. | Travel time, route cost, demand, vehicle availability, and reassignment consequences. |
| **Outcome and recovery evidence** | Business records establishing the effect of an API or GUI action, current approvals, unfinished obligations, and unresolved external updates. | Telemetry, production records, inspection results, or human confirmation establishing completed work, uncertain physical effects, and admissibility of dependent operations. | Reported command acceptance, route or service progress, confirmed completion, and uncertainty caused by stale state or missing confirmations. |
| **Applicability of later reuse** | Policy, application state, permissions, case conditions, and the accepted workflow or skill version. | Product, material, machine configuration, control software, quality tolerance, production policy, and affected procedure dependencies. | Traffic, location, communication regime, jurisdiction, fleet composition, service policy, and the accepted allocation, routing, or recovery version. |
| **Evaluation priorities** | Task success, output quality, completion time, rework, unresolved obligations at handoff or closure, responsibility transfer, and supervision cost. | Schedule feasibility, resource conflicts, recovery correctness and time, duplicated or unsafe actions, product quality, and reuse under changed conditions. | Allocation quality, commitment satisfaction, state freshness, coordination overhead, reassignment latency and effectiveness, handoff correctness, and robustness to interrupted communication. |

These prospective design profiles expand the domain implications in Section 7.2. Table A14 summarizes the information, evidence, and evaluation priorities for each setting. Actor status for the listed participants and components follows Section 3.1. The continuation and governed-change conditions in Sections 5.3 and 5.4 apply throughout.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



*Appendix E.1. Smart Productivity Workforce*

An enterprise work episode may span procurement, customer service, compliance, or internal approval, with interdependent tasks passing between people and software agents. WorkArena and WorkArena++ provide enterprise tasks and compositional workflows [102,103]. OSWorld includes work across computer applications [3], while WeaveBench examines workflows combining GUI interaction with CLI/code operations [4]. A deployment may similarly combine structured APIs with browser or desktop interaction in legacy systems.

Stable policies and reference records coexist with case-specific correspondence, intermediate analyses, approvals, and exceptions. Their provenance and versions determine which information supports an accepted decision. A handoff must retain unfinished obligations and unresolved actions alongside the current artifact state.

Organizational roles provide an access-control basis [50]. Operational delegation further determines who may approve a payment, waive a control, or close a case. Technical access to a record should therefore be accompanied by the applicable decision authority and escalation path. Evaluation must include responsibility transfer and human supervision cost, as well as the final business outcome.

*Appendix E.2. Adaptive Manufacturing*

Manufacturing couples computational decisions to physical production [104]. Machine failures, new jobs, and processing delays motivate dynamic rescheduling [105], while reconfigurable manufacturing treats responsiveness as a system-level concern [106]. Allocation must account for precedence, capability, capacity, setup, materials, work in progress, and commitments to other orders. Revising one operation can change downstream timing and resource feasibility.

Digital-twin distinctions draw attention to the data flows between physical and digital entities [107]. The production-management API, machine-control service, or legacy supervisory interface must therefore be related to the physical evidence it can provide. A digital dispatch record and an inspection result support different claims about production progress.

Material removal, assembly, and transfer can persist after software state is restored. Recovery requires reconciliation of physical work and uncertain issued actions before a dependent operation or retry proceeds. Machine-use authority, maintenance state, safety restrictions, and product-specific quality conditions constrain admissible recovery. Progression through inspection gates requires the applicable quality evidence. Later reuse must account for changes to machines, control software, materials, production policy, and tolerances, even when the task description is unchanged.

*Appendix E.3. Distributed Mobility*

Mobility processes span assignment, pickup or service commitments, routing, reassignment, and completion. Autonomous mobility-on-demand and dynamic vehicle-routing research address changing demand and network conditions [108,109]; multi-robot allocation provides a related treatment of assignments across available actors [69]. Reassignment changes both the receiving actor's work and the commitments attached to its previous allocation.

Vehicle-to-everything communication introduces delay, bandwidth limits, synchronization problems, and temporal asynchrony [110]. Observation timestamps, provenance, and reachability affect whether a remote state report still supports a decision when it is received. Implementations should specify how long remote information remains usable and when local operation must defer, degrade, or escalate.

Authority depends on role, controlled asset, region, and operating conditions. A fleet coordinator may allocate work while local safety constraints continue to govern its execution. A deployment may permit completion of an accepted route during disconnection while restricting new commitments outside the local delegation. Handoff to another vehicle, controller, coordinator, or human requires reconciliation of active commitments, uncertain command effects, and responsibility. Recovery and

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



reuse tests should vary traffic, location, communication regime, jurisdiction, fleet composition, and service policy.

**Author Contributions:** Quy Minh Le Conceptualization, Writing - original draft, Thai Hoa Writing - original draft, Tran Duc-Hai Nguyen Writing - original draft, Hoang D. Nguyen Conceptualization, Supervision, Writing - review and editing.

**Acknowledgments:** This publication has emanated from research conducted with the support of FPT Software and Taighde Éireann - Research Ireland (Grant number: 23/RC/13506). The UCC team is supported by Taighde Éireann - Research Ireland (Grant numbers: 12/RC/2289-P2, 18/CRT/6223 and 23/RC/13506). For the purpose of open access, the author has applied a CC BY public copyright licence to any Author Accepted Manuscript version arising from this submission.

## References

1. Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.R.; Cao, Y. ReAct: Synergizing Reasoning and Acting in Language Models. In Proceedings of the Proceedings of the Eleventh International Conference on Learning Representations, 2023.
2. Sumers, T.; Yao, S.; Narasimhan, K.R.; Griffiths, T.L. Cognitive Architectures for Language Agents. *Transactions on Machine Learning Research* **2024**. Survey Certification, Featured Certification.
3. Xie, T.; Zhang, D.; Chen, J.; Li, X.; Zhao, S.; Cao, R.; Hua, T.J.; Cheng, Z.; Shin, D.; Lei, F.; et al. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In Proceedings of the Advances in Neural Information Processing Systems, 2024, Vol. 37. Datasets and Benchmarks Track.
4. Li, W.; Zhou, B.; Yu, Y.; Xu, Z.; Yang, Y.; Li, D.; Shan, C. WeaveBench: A Long-Horizon, Real-World Benchmark for Computer-Use Agents with Hybrid Interfaces. *arXiv preprint arXiv:2606.09426* **2026**, [arXiv:cs.AI/2606.09426]. Accepted to the EMNLP 2026 Main Conference.
5. Hong, S.; Zhuge, M.; Chen, J.; Zheng, X.; Cheng, Y.; Zhang, C.; Wang, J.; Wang, Z.; Yau, S.K.S.; Lin, Z.; et al. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. In Proceedings of the Proceedings of the Twelfth International Conference on Learning Representations, 2024.
6. Wang, G.; Xie, Y.; Jiang, Y.; Mandlekar, A.; Xiao, C.; Zhu, Y.; Fan, L.; Anandkumar, A. Voyager: An Open-Ended Embodied Agent with Large Language Models. *Transactions on Machine Learning Research* **2024**.
7. Grosz, B.J.; Kraus, S. Collaborative Plans for Complex Group Action. *Artificial Intelligence* **1996**, *86*, 269–357. https://doi.org/10.1016/0004-3702(95)00103-4.
8. Tambe, M. Towards Flexible Teamwork. *Journal of Artificial Intelligence Research* **1997**, *7*, 83–124. https://doi.org/10.1613/jair.433.
9. Boella, G.; van der Torre, L.; Verhagen, H. Introduction to Normative Multiagent Systems. *Computational and Mathematical Organization Theory* **2006**, *12*, 71–79. https://doi.org/10.1007/s10588-006-9537-7.
10. Elnozahy, E.N.M.; Alvisi, L.; Wang, Y.M.; Johnson, D.B. A survey of rollback-recovery protocols in message-passing systems. *ACM Comput. Surv.* **2002**, *34*, 375–408. https://doi.org/10.1145/568522.568525.
11. Laird, J.E.; Newell, A.; Rosenbloom, P.S. SOAR: An Architecture for General Intelligence. *Artificial Intelligence* **1987**, *33*, 1–64. https://doi.org/10.1016/0004-3702(87)90050-6.
12. Hollan, J.; Hutchins, E.; Kirsh, D. Distributed Cognition: Toward a New Foundation for Human-Computer Interaction Research. *ACM Transactions on Computer-Human Interaction* **2000**, *7*, 174–196. https://doi.org/10.1145/353485.353487.
13. Zambonelli, F.; Jennings, N.R.; Wooldridge, M. Developing multiagent systems: The Gaia methodology. *ACM Trans. Softw. Eng. Methodol.* **2003**, *12*, 317–370. https://doi.org/10.1145/958961.958963.
14. Arcos, J.L.; Esteva, M.; Noriega, P.; Rodríguez-Aguilar, J.A.; Sierra, C. Engineering Open Environments with Electronic Institutions. *Engineering Applications of Artificial Intelligence* **2005**, *18*, 191–204. https://doi.org/10.1016/j.engappai.2004.11.019.
15. Hull, R.; Damaggio, E.; Fournier, F.; Gupta, M.; Fenno (Terry) Heath III.; Hobson, S.; Linehan, M.; Maradugu, S.; Nigam, A.; Sukaviriya, P.; et al. Introducing the Guard-Stage-Milestone Approach for Specifying Business Entity Lifecycles. In Proceedings of the Web Services and Formal Methods; Bravetti, M.; Bultan, T., Eds. Springer, 2011, Vol. 6551, *Lecture Notes in Computer Science*, pp. 1–24. https://doi.org/10.1007/978-3-642-19589-1_1.
16. Eshuis, R.; Hull, R.; Yi, M. Reasoning About Property Preservation in Adaptive Case Management. *ACM Trans. Internet Technol.* **2019**, *19*. https://doi.org/10.1145/3177778.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1




17. White, S.R.; Hanson, J.E.; Whalley, I.; Chess, D.M.; Kephart, J.O. An Architectural Approach to Autonomic Computing. In Proceedings of the Proceedings of the International Conference on Autonomic Computing (ICAC 2004). IEEE, 2004, pp. 2–9. https://doi.org/10.1109/ICAC.2004.1301340.
18. IBM Corporation. An Architectural Blueprint for Autonomic Computing. White paper, IBM Corporation, 2005. Third Edition.
19. Burrows, M. The Chubby Lock Service for Loosely-Coupled Distributed Systems. In Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI 06), Seattle, WA, 2006.
20. van der Aalst, W.M.P.; ter Hofstede, A.H.M.; Kiepuszewski, B.; Barros, A.P. Workflow Patterns. *Distributed and Parallel Databases* **2003**, *14*, 5–51. https://doi.org/10.1023/A:1022883727209.
21. van der Aalst, W.M.P. Process Mining: Overview and Opportunities. *ACM Transactions on Management Information Systems* **2012**, *3*, 1–17. https://doi.org/10.1145/2229156.2229157.
22. Parisi, G.I.; Kemker, R.; Part, J.L.; Kanan, C.; Wermter, S. Continual Lifelong Learning with Neural Networks: A Review. *Neural Networks* **2019**, *113*, 54–71. https://doi.org/10.1016/j.neunet.2019.01.012.
23. Zhou, A.; Yan, K.; Shlapentokh-Rothman, M.; Wang, H.; Wang, Y.X. Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models. In Proceedings of the Proceedings of the 41st International Conference on Machine Learning. PMLR, 2024, Vol. 235, *Proceedings of Machine Learning Research*, pp. 62138–62160.
24. Wu, Q.; Bansal, G.; Zhang, J.; Wu, Y.; Li, B.; Zhu, E.; Jiang, L.; Zhang, X.; Zhang, S.; Liu, J.; et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. In Proceedings of the Conference on Language Modeling (COLM), August 2024.
25. Li, G.; Hammoud, H.; Itani, H.; Khizbullin, D.; Ghanem, B. CAMEL: Communicative Agents for "Mind" Exploration of Large Language Model Society. In Proceedings of the Advances in Neural Information Processing Systems, 2023, Vol. 36. https://doi.org/10.52202/075280-2264.
26. Yao, S.; Shinn, N.; Razavi, P.; Narasimhan, K. $\tau$-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. In Proceedings of the Proceedings of the Thirteenth International Conference on Learning Representations, 2025.
27. Debenedetti, E.; Zhang, J.; Balunović, M.; Beurer-Kellner, L.; Fischer, M.; Tramèr, F. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In Proceedings of the Advances in Neural Information Processing Systems, 2024, Vol. 37. Datasets and Benchmarks Track.
28. Kapoor, S.; Stroebl, B.; Siegel, Z.S.; Nadgir, N.; Narayanan, A. AI Agents That Matter. *Transactions on Machine Learning Research* **2025**.
29. Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; Yao, S. Reflexion: Language Agents with Verbal Reinforcement Learning. In Proceedings of the Advances in Neural Information Processing Systems, 2023, Vol. 36. https://doi.org/10.52202/075280-0377.
30. Wang, Z.Z.; Mao, J.; Fried, D.; Neubig, G. Agent Workflow Memory. In Proceedings of the Proceedings of the 42nd International Conference on Machine Learning; Singh, A.; Fazel, M.; Hsu, D.; Lacoste-Julien, S.; Berkenkamp, F.; Maharaj, T.; Wagstaff, K.; Zhu, J., Eds. PMLR, 13–19 Jul 2025, Vol. 267, *Proceedings of Machine Learning Research*, pp. 63897–63911.
31. Xu, W.; Liang, Z.; Mei, K.; Gao, H.; Tan, J.; Zhang, Y. A-Mem: Agentic Memory for LLM Agents. In Proceedings of the Advances in Neural Information Processing Systems, 2025, Vol. 38.
32. Jin, B.; Zeng, H.; Yue, Z.; Yoon, J.; Arik, S.; Wang, D.; Zamani, H.; Han, J. Search-R1: Training LLMs to Reason and Leverage Search Engines with Reinforcement Learning. In Proceedings of the Proceedings of the Second Conference on Language Modeling, 2025.
33. Zhang, G.; Geng, H.; Yu, X.; Yin, Z.; Zhang, Z.; Tan, Z.; et al. The Landscape of Agentic Reinforcement Learning for LLMs: A Survey. *Transactions on Machine Learning Research* **2026**.
34. Khan, S. Resume Means Resume: A Machine-Checked Conformance Contract for Checkpoint, Interrupt, and Resume Semantics in Workflow Persistence Layers. *arXiv preprint arXiv:2608.03836* **2026**. https://doi.org/10.48550/arXiv.2608.03836.
35. Wu, G.; Li, D.; Jiang, K.; Niu, J.; Wang, C.; Zhang, Y. Safe to Resume? Breaking Execution Continuity of Agent Execution via Rollback. *arXiv preprint arXiv:2608.29381* **2026**. https://doi.org/10.48550/arXiv.2608.29381.
36. Mohammadi, B.; Potamitis, N.; Klein, L.; Arora, A.; Bindschaedler, L. Atomix: Timely, Transactional Tool Use for Reliable Agentic Workflows. *arXiv preprint arXiv:2602.14849* **2026**. https://doi.org/10.48550/arXiv.2602.14849.


© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1




37. Samanta, R.; Saha, B.; Ghosh, S.K.; Buyya, R. AgentR: A Stateful and Recovery-Aware Software Architecture for LLM-based Auditable Workflows. *arXiv preprint arXiv:2608.15264* **2026**. https://doi.org/10.48550/arXiv.2608.15264.
38. Filho, E.B.d.S. ESAA-Conversational: An Event-Sourced Memory Layer for Continuity, Handoff, and Curation Across Heterogeneous LLM Coding Agents. *arXiv preprint arXiv:2606.23752* **2026**. https://doi.org/10.48550/arXiv.2606.23752.
39. Ding, T.; Nannapaneni, A.; Liu, B.; Zhang, L. Always-On Agents: A Survey of Persistent Memory, State, and Governance in LLM Agents. *arXiv preprint arXiv:2606.30306* **2026**. https://doi.org/10.48550/arXiv.2606.30306.
40. Khanal, A.; Tao, Y.; Zhou, J. Beyond pass@1: A Reliability Science Framework for Long-Horizon LLM Agents. *arXiv preprint arXiv:2603.29231* **2026**. https://doi.org/10.48550/arXiv.2603.29231.
41. Cemri, M.; Pan, M.Z.; Yang, S.; Agrawal, L.A.; Chopra, B.; Tiwari, R.; Keutzer, K.; Parameswaran, A.; Klein, D.; Ramchandran, K.; et al. Why Do Multi-Agent LLM Systems Fail? In Proceedings of the Advances in Neural Information Processing Systems, 2025, Vol. 38. Datasets and Benchmarks Track.
42. Rabanser, S.; Kapoor, S.; Kirgis, P.; Liu, K.; Utpala, S.; Narayanan, A. Towards a Science of AI Agent Reliability. In Proceedings of the Proceedings of the 43rd International Conference on Machine Learning; Zhang, T.; Dudik, M.; Jaggi, M.; Agarwal, A.; Li, S.; Schuurmans, D.; Zhu, J.; Berkenkamp, F.; Dong, H.; Bietti, A., Eds. PMLR, 06–11 Jul 2026, Vol. 306, *Proceedings of Machine Learning Research*, pp. 102769–102820.
43. Mei, K.; Zhu, X.; Xu, W.; Jin, M.; Hua, W.; Li, Z.; Xu, S.; Ye, R.; Ge, Y.; Zhang, Y. AIOS: LLM Agent Operating System. In Proceedings of the Proceedings of the Second Conference on Language Modeling, 2025.
44. Verma, A.; Pedrosa, L.; Korupolu, M.; Oppenheimer, D.; Tune, E.; Wilkes, J. Large-Scale Cluster Management at Google with Borg. In Proceedings of the Proceedings of the Tenth European Conference on Computer Systems, 2015, pp. 1–17. https://doi.org/10.1145/2741948.2741964.
45. Anthropic. Model Context Protocol Specification. Technical specification, 2025. Accessed: 13 Sep. 2026.
46. Linux Foundation. Agent2Agent (A2A) Protocol Specification. Technical specification, 2025. Accessed: 13 Sep. 2026.
47. Chan, A.; Wei, K.; Huang, S.; Rajkumar, N.; Perrier, E.; Lazar, S.; Hadfield, G.K.; Anderljung, M. Infrastructure for AI Agents. *Transactions on Machine Learning Research* **2025**.
48. Sharma, A.; Shah, D. The Agent Operating System (AOS): A Reference Operating Architecture for Distributed Agentic Systems. *arXiv preprint arXiv:2608.03214* **2026**. https://doi.org/10.48550/arXiv.2608.03214.
49. Meyer, B. Applying "Design by Contract". *Computer* **1992**, *25*, 40–51. https://doi.org/10.1109/2.161279.
50. Sandhu, R.S.; Coyne, E.J.; Feinstein, H.L.; Youman, C.E. Role-Based Access Control Models. *Computer* **1996**, *29*, 38–47. https://doi.org/10.1109/2.485845.
51. Saltzer, J.H.; Schroeder, M.D. The Protection of Information in Computer Systems. *Proceedings of the IEEE* **1975**, *63*, 1278–1308. https://doi.org/10.1109/PROC.1975.9939.
52. Wang, H.; Poskitt, C.M.; Sun, J. AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents. In Proceedings of the Proceedings of the 48th IEEE/ACM International Conference on Software Engineering, 2026, pp. 2938–2950. Research Track, https://doi.org/10.1145/3744916.3764546.
53. Xiang, Z.; Zheng, L.; Li, Y.; Hong, J.; Li, Q.; Xie, H.; Zhang, J.; Xiong, Z.; Xie, C.; Yang, C.; et al. GuardAgent: Safeguard LLM Agents via Knowledge-Enabled Reasoning. In Proceedings of the Proceedings of the 42nd International Conference on Machine Learning; Singh, A.; Fazel, M.; Hsu, D.; Lacoste-Julien, S.; Berkenkamp, F.; Maharaj, T.; Wagstaff, K.; Zhu, J., Eds. PMLR, 13–19 Jul 2025, Vol. 267, *Proceedings of Machine Learning Research*, pp. 68316–68342.
54. Bauer, A.; Leucker, M.; Schallhart, C. Runtime Verification for LTL and TLTL. *ACM Transactions on Software Engineering and Methodology* **2011**, *20*, 14:1–14:64. https://doi.org/10.1145/2000799.2000800.
55. Saltzer, J.H.; Reed, D.P.; Clark, D.D. End-to-End Arguments in System Design. *ACM Transactions on Computer Systems* **1984**, *2*, 277–288. https://doi.org/10.1145/357401.357402.
56. Herschel, M.; Diestelkämper, R.; Ben Lahmar, H. A Survey on Provenance: What For? What Form? What From? *The VLDB Journal* **2017**, *26*, 881–906. https://doi.org/10.1007/s00778-017-0486-1.
57. Chan, A.; Ezell, C.; Kaufmann, M.; Wei, K.; Hammond, L.; Bradley, H.; Bluemke, E.; Rajkumar, N.; Krueger, D.; Kolt, N.; et al. Visibility into AI Agents. In Proceedings of the Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency, 2024, pp. 958–973. https://doi.org/10.1145/3630106.3658948.
58. Thurlow, R. RPC: Remote Procedure Call Protocol Specification Version 2. RFC 5531, RFC Editor, 2009. https://doi.org/10.17487/RFC5531.


© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1




59. Mitchell, M.; Wu, S.; Zaldivar, A.; Barnes, P.; Vasserman, L.; Hutchinson, B.; Spitzer, E.; Raji, I.D.; Gebru, T. Model Cards for Model Reporting. In Proceedings of the Proceedings of the Conference on Fairness, Accountability, and Transparency. Association for Computing Machinery, 2019, pp. 220–229. https://doi.org/10.1145/3287560.3287596.
60. Sculley, D.; Holt, G.; Golovin, D.; Davydov, E.; Phillips, T.; Ebner, D.; Chaudhary, V.; Young, M.; Crespo, J.F.; Dennison, D. Hidden Technical Debt in Machine Learning Systems. In Proceedings of the Advances in Neural Information Processing Systems, 2015, Vol. 28, pp. 2503–2511.
61. Garcia-Molina, H.; Salem, K. Sagas. In Proceedings of the Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data, 1987, pp. 249–259. https://doi.org/10.1145/38713.38742.
62. MetaGPT contributors. MetaGPT v0.8.2: Source Code, 2025. Pinned source revision. Accessed: Sep. 29, 2026.
63. LangChain contributors. Interrupts: Interrupts in Tools, Python Full Example, 2026. Pinned documentation source, lines 593–693. Accessed: Sep. 29, 2026.
64. LangChain contributors. LangGraph 1.2.11: Source Code, 2026. Pinned runtime source revision. Accessed: Sep. 29, 2026.
65. Voyager contributors. Voyager: Source Code, 2023. Pinned source revision. Accessed: Sep. 29, 2026.
66. AIOS contributors. AIOS v0.2.2: Source Code, 2025. Pinned source revision. Accessed: Sep. 29, 2026.
67. Jabareen, Y. Building a Conceptual Framework: Philosophy, Definitions, and Procedure. *International Journal of Qualitative Methods* **2009**, *8*, 49–62. https://doi.org/10.1177/160940690900800406.
68. Nickerson, R.C.; Varshney, U.; Muntermann, J. A Method for Taxonomy Development and Its Application in Information Systems. *European Journal of Information Systems* **2013**, *22*, 336–359. https://doi.org/10.1057/ejis.2012.26.
69. Gerkey, B.P.; Matarić, M.J. A Formal Analysis and Taxonomy of Task Allocation in Multi-Robot Systems. *The International Journal of Robotics Research* **2004**, *23*, 939–954. https://doi.org/10.1177/0278364904045564.
70. Fioretto, F.; Pontelli, E.; Yeoh, W. Distributed Constraint Optimization Problems and Applications: A Survey. *Journal of Artificial Intelligence Research* **2018**, *61*, 623–698. https://doi.org/10.1613/jair.5565.
71. Stern, R.; Sturtevant, N.R.; Felner, A.; Koenig, S.; Ma, H.; Walker, T.T.; Li, J.; Atzmon, D.; Cohen, L.; Kumar, T.K.S.; et al. Multi-Agent Pathfinding: Definitions, Variants, and Benchmarks. In Proceedings of the Proceedings of the 12th Annual Symposium on Combinatorial Search, 2019, Vol. 10, pp. 151–158. https://doi.org/10.1609/socs.v10i1.18510.
72. Park, J.S.; O'Brien, J.; Cai, C.J.; Morris, M.R.; Liang, P.; Bernstein, M.S. Generative Agents: Interactive Simulacra of Human Behavior. In Proceedings of the Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology, New York, NY, USA, 2023; UIST '23. https://doi.org/10.1145/3586183.3606763.
73. Kaelbling, L.P.; Littman, M.L.; Cassandra, A.R. Planning and Acting in Partially Observable Stochastic Domains. *Artificial Intelligence* **1998**, *101*, 99–134. https://doi.org/10.1016/S0004-3702(98)00023-X.
74. Farquhar, S.; Kossen, J.; Kuhn, L.; Gal, Y. Detecting Hallucinations in Large Language Models Using Semantic Entropy. *Nature* **2024**, *630*, 625–630. https://doi.org/10.1038/s41586-024-07421-0.
75. Ichter, B.; Brohan, A.; Chebotar, Y.; Finn, C.; Hausman, K.; Herzog, A.; Ho, D.; Ibarz, J.; Irpan, A.; Jang, E.; et al. Do As I Can, Not As I Say: Grounding Language in Robotic Affordances. In Proceedings of the Proceedings of The 6th Conference on Robot Learning; Liu, K.; Kulic, D.; Ichnowski, J., Eds. PMLR, 14–18 Dec 2023, Vol. 205, *Proceedings of Machine Learning Research*, pp. 287–318.
76. Vasconcelos, W.W.; García-Camino, A.; Gaertner, D.; Rodríguez-Aguilar, J.A.; Noriega, P. Distributed Norm Management for Multi-Agent Systems. *Expert Systems with Applications* **2012**, *39*, 5990–5999. https://doi.org/10.1016/j.eswa.2011.11.108.
77. Schierman, J.D.; DeVore, M.D.; Richards, N.D.; Clark, M.A. Runtime Assurance for Autonomous Aerospace Systems. *Journal of Guidance, Control, and Dynamics* **2020**, *43*, 2205–2217. https://doi.org/10.2514/1.G004862.
78. Sifakis, J.; Harel, D. Trustworthy Autonomous System Development. *ACM Trans. Embed. Comput. Syst.* **2023**, *22*. https://doi.org/10.1145/3545178.
79. Hevner, A.R.; March, S.T.; Park, J.; Ram, S. Design Science in Information Systems Research. *Management Information Systems Quarterly* **2004**, *28*, 75–106. https://doi.org/10.2307/25148625.
80. Hayes, A.F.; Krippendorff, K. Answering the Call for a Standard Reliability Measure for Coding Data. *Communication Methods and Measures* **2007**, *1*, 77–89. https://doi.org/10.1080/19312450709336664.


© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1




81. Liu, X.; Yu, H.; Zhang, H.; Xu, Y.; Lei, X.; Lai, H.; Gu, Y.; Ding, H.; Men, K.; Yang, K.; et al. AgentBench: Evaluating LLMs as Agents. In Proceedings of the International Conference on Learning Representations; Kim, B.; Yue, Y.; Chaudhuri, S.; Fragkiadaki, K.; Khan, M.; Sun, Y., Eds., 2024, Vol. 2024, pp. 52989–53046.
82. Venable, J.; Pries-Heje, J.; Baskerville, R. FEDS: A Framework for Evaluation in Design Science Research. *European Journal of Information Systems* **2016**, *25*, 77–89. https://doi.org/10.1057/ejis.2014.36.
83. Baltrušaitis, T.; Ahuja, C.; Morency, L.P. Multimodal Machine Learning: A Survey and Taxonomy. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **2019**, *41*, 423–443. https://doi.org/10.1109/TPAMI.2018.2798607.
84. Akidau, T.; Bradshaw, R.; Chambers, C.; Chernyak, S.; Fernández-Moctezuma, R.J.; Lax, R.; McVeety, S.; Mills, D.; Perry, F.; Schmidt, E.; et al. The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing. *Proceedings of the VLDB Endowment* **2015**, *8*, 1792–1803. https://doi.org/10.14778/2824032.2824076.
85. Moreau, L.; Missier, P. PROV-DM: The PROV Data Model. W3C recommendation, World Wide Web Consortium, 2013.
86. Koh, J.Y.; Lo, R.; Jang, L.; Duvvur, V.; Lim, M.C.; Huang, P.Y.; Neubig, G.; Zhou, S.; Salakhutdinov, R.; Fried, D. VisualWebArena: Evaluating Multimodal Agents on Realistic Visually Grounded Web Tasks. In Proceedings of the Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Association for Computational Linguistics, 2024, pp. 881–905. https://doi.org/10.18653/v1/2024.acl-long.50.
87. Hu, Y.; Shi, W.; Fu, X.; Roth, D.; Ostendorf, M.; Zettlemoyer, L.; Smith, N.A.; Krishna, R. Visual Sketchpad: Sketching as a Visual Chain of Thought for Multimodal Language Models. In Proceedings of the Advances in Neural Information Processing Systems, 2024, Vol. 37. https://doi.org/10.52202/079017-4423.
88. Zhang, Y.; Zou, Y.; Zhang, S.; Hu, J.; Zhang, H.; Xu, B.; Kautz, J.; Dong, Y. Agora: Git as Shared Memory for Collective AutoResearch, 2026, [arXiv:cs.LG/2609.18094]. arXiv preprint arXiv:2609.18094v1, https://doi.org/10.48550/arXiv.2609.18094.
89. Gruber, T.R. A Translation Approach to Portable Ontology Specifications. *Knowledge Acquisition* **1993**, *5*, 199–220. https://doi.org/10.1006/knac.1993.1008.
90. Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Küttler, H.; Lewis, M.; Yih, W.t.; Rocktäschel, T.; et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In Proceedings of the Proceedings of the 34th International Conference on Neural Information Processing Systems, Red Hook, NY, USA, 2020; NIPS '20.
91. Hafner, D.; Pasukonis, J.; Ba, J.; Lillicrap, T. Mastering Diverse Control Tasks through World Models. *Nature* **2025**, *640*, 647–653. https://doi.org/10.1038/s41586-025-08744-2.
92. Schick, T.; Dwivedi-Yu, J.; Dessí, R.; Raileanu, R.; Lomeli, M.; Hambro, E.; Zettlemoyer, L.; Cancedda, N.; Scialom, T. Toolformer: language models can teach themselves to use tools. In Proceedings of the Proceedings of the 37th International Conference on Neural Information Processing Systems, Red Hook, NY, USA, 2023; NIPS '23.
93. Alshiekh, M.; Bloem, R.; Ehlers, R.; Könighofer, B.; Niekum, S.; Topcu, U. Safe reinforcement learning via shielding. In Proceedings of the Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence. AAAI Press, 2018, AAAI'18/IAAI'18/EAAI'18.
94. Zhao, A.; Huang, D.; Xu, Q.; Lin, M.; Liu, Y.J.; Huang, G. ExpeL: LLM Agents Are Experiential Learners. In Proceedings of the Proceedings of the AAAI Conference on Artificial Intelligence, 2024, Vol. 38, pp. 19632–19642. https://doi.org/10.1609/aaai.v38i17.29936.
95. Bailis, P.; Fekete, A.; Franklin, M.J.; Ghodsi, A.; Hellerstein, J.M.; Stoica, I. Coordination avoidance in database systems. *Proc. VLDB Endow.* **2014**, *8*, 185–196. https://doi.org/10.14778/2735508.2735509.
96. Huang, J.; Chen, X.; Mishra, S.; Zheng, H.S.; Yu, A.W.; Song, X.; Zhou, D. Large Language Models Cannot Self-Correct Reasoning Yet. In Proceedings of the Proceedings of the Twelfth International Conference on Learning Representations, 2024.
97. Clarke, E.M.; Emerson, E.A.; Sistla, A.P. Automatic Verification of Finite-State Concurrent Systems Using Temporal Logic Specifications. *ACM Transactions on Programming Languages and Systems* **1986**, *8*, 244–263. https://doi.org/10.1145/5397.5399.
98. Zhai, Z.; Li, M.; Wang, X. Revisable by Design: A Theory of Streaming LLM Agent Execution. *arXiv preprint arXiv:2604.23283* **2026**. https://doi.org/10.48550/arXiv.2604.23283.
99. Mohammadi, B.; Bindschaedler, L. The Irreversibility Budget: Fleet-Level Risk Accounting and Admission Control for Agent Operating Systems. *arXiv preprint arXiv:2609.00275* **2026**. Accepted to the 2nd Workshop


© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 10 October 2026 doi:10.20944/preprints202610.0802.v1



on Operating Systems Design for AI Agents (AgenticOS 2026), a non-archival workshop co-located with SOSP 2026, https://doi.org/10.48550/arXiv.2609.00275.

100. LangChain contributors. LangGraph Checkpoint 4.2.0: Source Code, 2026. Pinned checkpoint source revision. Accessed: Sep. 29, 2026.
101. LangChain contributors. LangGraph Checkpoint SQLite 3.1.1, 2026. Inspected artifact: langgraph_checkpoint_sqlite-3.1.1-py3-none-any.whl. Accessed: Sep. 29, 2026.
102. Drouin, A.; Gasse, M.; Caccia, M.; Laradji, I.H.; Del Verme, M.; Marty, T.; Vazquez, D.; Chapados, N.; Lacoste, A. WorkArena: How Capable are Web Agents at Solving Common Knowledge Work Tasks? In Proceedings of the Proceedings of the 41st International Conference on Machine Learning; Salakhutdinov, R.; Kolter, Z.; Heller, K.; Weller, A.; Oliver, N.; Scarlett, J.; Berkenkamp, F., Eds. PMLR, 21–27 Jul 2024, Vol. 235, *Proceedings of Machine Learning Research*, pp. 11642–11662.
103. Boisvert, L.; Thakkar, M.; Gasse, M.; Caccia, M.; De Chezelles, T.L.S.; Cappart, Q.; Chapados, N.; Lacoste, A.; Drouin, A. WorkArena++: Towards Compositional Planning and Reasoning-based Common Knowledge Work Tasks. In Proceedings of the Advances in Neural Information Processing Systems; Globerson, A.; Mackey, L.; Belgrave, D.; Fan, A.; Paquet, U.; Tomczak, J.; Zhang, C., Eds. Curran Associates, Inc., 2024, Vol. 37, pp. 5996–6051. https://doi.org/10.52202/079017-0195.
104. Monostori, L.; Kádár, B.; Bauernhansl, T.; Kondoh, S.; Kumara, S.; Reinhart, G.; Sauer, O.; Schuh, G.; Sihn, W.; Ueda, K. Cyber-physical systems in manufacturing. *CIRP Annals* **2016**, *65*, 621–641. https://doi.org/10.1016/j.cirp.2016.06.005.
105. Ouelhadj, D.; Petrovic, S. A Survey of Dynamic Scheduling in Manufacturing Systems. *Journal of Scheduling* **2009**, *12*, 417–431. https://doi.org/10.1007/s10951-008-0090-8.
106. Koren, Y.; Heisel, U.; Jovane, F.; Moriwaki, T.; Pritschow, G.; Ulsoy, G.; Van Brussel, H. Reconfigurable Manufacturing Systems. *CIRP Annals* **1999**, *48*, 527–540. https://doi.org/10.1016/S0007-8506(07)63232-6.
107. Kritzinger, W.; Karner, M.; Traar, G.; Henjes, J.; Sihn, W. Digital Twin in manufacturing: A categorical literature review and classification. *IFAC-PapersOnLine* **2018**, *51*, 1016–1022. 16th IFAC Symposium on Information Control Problems in Manufacturing INCOM 2018, https://doi.org/10.1016/j.ifacol.2018.08.474.
108. Zardini, G.; Lanzetti, N.; Pavone, M.; Frazzoli, E. Analysis and Control of Autonomous Mobility-on-Demand Systems. *Annual Review of Control, Robotics, and Autonomous Systems* **2022**, *5*, 633–658. https://doi.org/10.1146/annurev-control-042920-012811.
109. Psaraftis, H.N.; Wen, M.; Kontovas, C.A. Dynamic vehicle routing problems: Three decades and counting. *Networks* **2016**, *67*, 3–31, [https://onlinelibrary.wiley.com/doi/pdf/10.1002/net.21628]. https://doi.org/10.1002/net.21628.
110. Zhang, X.; Li, J.; Zhou, J.; Zhang, S.; Wang, J.; Yuan, Y.; Liu, J.; Li, J. Vehicle-to-Everything Communication in Intelligent Connected Vehicles: A Survey and Taxonomy. *Automotive Innovation* **2025**, *8*, 13–45. https://doi.org/10.1007/s42154-024-00310-2.

**Disclaimer/Publisher's Note:** The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.